

fbfrontend - Phase 9.1-9.3

Account state and error feedback

Complete source files and before/after changes

22 files: 19 updated and 3 added. Implemented locally on top of Phase 8.4.

9.1: Saved profile and image responses update shared account state. Drafts remain independent; overlapping mutations and stale responses are guarded.

9.2: Shared backend error formatting and accessible inline field errors. Existing payloads and endpoints are unchanged; uncertain-send feedback is preserved.

9.3: Same-tab/cross-tab auth events, session-scoped requests, stale refresh protection and retryable temporary outages. Backend authorization remains authoritative.

Verified: 28 unit tests; new account-state browser checks; existing auth, chat, sidebar and UI interaction regressions. Production build passed. No lint errors; 13 warnings remain.

Mocked local services only. Backend and FrndBookWC2 unchanged. No cloud data changes, migration, dependency or deployment.

Every file has full current source and a unified diff against Phase 8.4. Green + lines are added; red - lines are removed. Gutter numbers are aids; long lines wrap with >.

The companion Plan and Testing Markdown provides detailed decisions, manual checks, test commands and limitations. Cross-tab storage is not a distributed refresh lock or a new backend security mechanism.

File index

src/api/axiosClient.js	3
src/auth/AuthContext.jsx	8
src/auth/ProtectedRoute.jsx	14
src/auth/PublicRoute.jsx	16
src/components/users/UserSearch.jsx	18
src/context/NotificationContext.jsx	24
src/pages/ForgotPassword.jsx	31
src/pages/FriendRequests.jsx	34
src/pages/Friends.jsx	38
src/pages/Login.jsx	42
src/pages/Messages.jsx	45
src/pages/Notifications.jsx	50
src/pages/Profile.jsx	55
src/pages/ResetPassword.jsx	63
src/pages/Signup.jsx	66
src/pages/UserProfile.jsx	69
src/pages/VerifyEmail.jsx	76
src/services/messageSession.js	81
src/utils/apiError.js	85
src/utils/tokenStorage.js	87
tests/accountState.browser.mjs	90
tests/apiError.test.js	94

```

1  import axios from "axios";
2
3  import { tokenStorage } from "../utils/tokenStorage";
4
5  const axiosClient = axios.create({
6    baseURL: import.meta.env.VITE_API_BASE_URL,
7
8    // ISSUE FIX:
9    // Do not set Content-Type globally.
10   // Profile image uploads use FormData and must be sent as
11   // multipart/form-data with the boundary generated automatically.
12 });
13
14 // =====
15 // REFRESH STATE
16 // =====
17
18 let refreshFlight = null;
19
20 export const refreshAccessToken = () => {
21   const refreshToken = tokenStorage.getRefreshToken();
22   const session = tokenStorage.getSessionId();
23   if (!refreshToken) return Promise.reject(new Error("Refresh token is not available"));
24   if (refreshFlight?.token === refreshToken && refreshFlight.session === session) return refreshFlight.promise;
25   const flight = { token: refreshToken, session };
26   const stillCurrent = () => tokenStorage.getRefreshToken() === refreshToken && tokenStorage.getSessionId() === s
27   > session;
28   flight.promise = axios.post(`${import.meta.env.VITE_API_BASE_URL}/api/auth/refresh`, { refreshToken })
29     .then(response => {
30       if (!stillCurrent()) throw new Error("Authentication changed while refreshing");
31       const { accessToken, refreshToken: newRefreshToken } = response.data;
32       if (!accessToken) throw new Error("Invalid refresh response");
33       tokenStorage.updateTokens(accessToken, newRefreshToken);
34       return accessToken;
35     })
36     .catch(error => {
37       // A temporary outage is not evidence that a session has been revoked.
38       if (stillCurrent() && [401, 403].includes(error.response?.status)) tokenStorage.clear();
39       throw error;
40     })
41     .finally(() => { if (refreshFlight === flight) refreshFlight = null; });
42   refreshFlight = flight;
43   return flight.promise;
44 };
45
46 // =====
47 // REQUEST INTERCEPTOR
48 // =====
49
50 axiosClient.interceptors.request.use(
51   (config) => {
52     const session = tokenStorage.getSessionId();
53     if (config._sessionCaptured && config._authSession !== session) {
54       return Promise.reject(new axios.CancelError("Session changed"));
55     }
56     config._sessionCaptured = true;
57     config._authSession = session;
58     const accessToken = tokenStorage.getAccessToken();
59
60     if (accessToken) {
61       config.headers.Authorization = `Bearer ${accessToken}`;
62     }
63
64     return config;
65   },
66   (error) => {
67     return Promise.reject(error);
68   },
69 );

```

```

70
71 // =====
72 // RESPONSE INTERCEPTOR
73 // =====
74
75 axiosClient.interceptors.response.use(
76   (response) => {
77     if (response.config._authSession !== tokenStorage.getSessionId()) throw new axios.CanceledError("Session chan
78   > ged");
79     return response;
80   },
81   async (error) => {
82     const originalRequest = error.config;
83     if (!originalRequest || (originalRequest._sessionCaptured && originalRequest._authSession !== tokenStorage.ge
84   > tSessionId())) return Promise.reject(error);
85
86     // -----
87     // Only handle 401
88     // -----
89     if (error.response?.status !== 401) {
90       return Promise.reject(error);
91     }
92
93     // -----
94     // Prevent infinite retry
95     // -----
96
97     if (originalRequest?._retry) {
98       return Promise.reject(error);
99     }
100
101     // -----
102     // Never refresh these endpoints
103     // -----
104
105     const requestUrl = originalRequest?.url || "";
106
107     /*
108     * ISSUE FIX:
109     * All public authentication endpoints must bypass
110     * the access-token refresh flow.
111     */
112     if (
113       requestUrl.includes("/api/auth/login") ||
114       requestUrl.includes("/api/auth/signup") ||
115       requestUrl.includes("/api/auth/verify-email") ||
116       requestUrl.includes("/api/auth/resend-verification") ||
117       requestUrl.includes("/api/auth/refresh") ||
118       requestUrl.includes("/api/auth/forgot-password") ||
119       requestUrl.includes("/api/auth/reset-password")
120     ) {
121       return Promise.reject(error);
122     }
123
124     // -----
125     // Mark request
126     // -----
127
128     originalRequest._retry = true;
129
130     try {
131       const newAccessToken = await refreshAccessToken();
132
133       originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
134
135       return axiosClient(originalRequest);
136     } catch (refreshError) {
137       /*

```

src/api/axiosClient.js

FULL CURRENT CONTENT

```
138         * Refresh failed completely.
139         * The shared refresh function has already
140         * cleared authentication.
141         */
142
143         // AuthContext reacts to auth-cleared; transient failures stay retryable.
144
145         return Promise.reject(refreshError);
146     }
147 },
148 );
149
150 export default axiosClient;
```

```

1  --- before/src/api/axiosClient.js
2  +++ after/src/api/axiosClient.js
3  @@ -15,51 +15,31 @@
4  // REFRESH STATE
5  // =====
6
7  -let refreshPromise = null;
8  +let refreshFlight = null;
9
10 -// =====
11 -// REFRESH ACCESS TOKEN
12 -// =====
13 -
14 -export const refreshAccessToken = async () => {
15 - /*
16 -  * If another refresh is already running,
17 -  * everyone waits for that same request.
18 -  */
19 - if (refreshPromise) {
20 -   return refreshPromise;
21 - }
22 -
23 +export const refreshAccessToken = () => {
24   const refreshToken = tokenStorage.getRefreshToken();
25 -
26 - if (!refreshToken) {
27 -   throw new Error("Refresh token is not available");
28 - }
29 -
30 - refreshPromise = axios
31 -   .post(`${import.meta.env.VITE_API_BASE_URL}/api/auth/refresh`, {
32 -     refreshToken,
33 -   })
34 -   .then((response) => {
35 -     if (tokenStorage.getRefreshToken() !== refreshToken) {
36 -       throw new Error("Authentication changed while refreshing");
37 -     }
38 + const session = tokenStorage.getSessionId();
39 + if (!refreshToken) return Promise.reject(new Error("Refresh token is not available"));
40 + if (refreshFlight?.token === refreshToken && refreshFlight.session === session) return refreshFlight.promise;
41 + const flight = { token: refreshToken, session };
42 + const stillCurrent = () => tokenStorage.getRefreshToken() === refreshToken && tokenStorage.getSessionId() ===
43   > session;
44 + flight.promise = axios.post(`${import.meta.env.VITE_API_BASE_URL}/api/auth/refresh`, { refreshToken })
45 +   .then(response => {
46 +     if (!stillCurrent()) throw new Error("Authentication changed while refreshing");
47 +     const { accessToken, refreshToken: newRefreshToken } = response.data;
48 +     if (!accessToken) throw new Error("Invalid refresh response");
49 +     tokenStorage.updateTokens(accessToken, newRefreshToken);
50 -
51 -     return accessToken;
52 -   })
53 -   .catch((error) => {
54 -     if (tokenStorage.getRefreshToken() === refreshToken) tokenStorage.clear();
55 -
56 +     .catch(error => {
57 +       // A temporary outage is not evidence that a session has been revoked.
58 +       if (stillCurrent() && [401, 403].includes(error.response?.status)) tokenStorage.clear();
59       throw error;
60     })
61 -   .finally(() => {
62 -     refreshPromise = null;
63 -   });
64 -
65 - return refreshPromise;
66 +   .finally(() => { if (refreshFlight === flight) refreshFlight = null; });
67 + refreshFlight = flight;
68 + return flight.promise;
69   };

```

```

70
71 // =====
72 @@ -68,6 +48,12 @@
73
74 axiosClient.interceptors.request.use(
75   (config) => {
76 +   const session = tokenStorage.getSessionId();
77 +   if (config._sessionCaptured && config._authSession !== session) {
78 +     return Promise.reject(new axios.CanceledError("Session changed"));
79 +   }
80 +   config._sessionCaptured = true;
81 +   config._authSession = session;
82   const accessToken = tokenStorage.getAccessToken();
83
84   if (accessToken) {
85 @@ -88,11 +74,13 @@
86
87 axiosClient.interceptors.response.use(
88   (response) => {
89 +   if (response.config._authSession !== tokenStorage.getSessionId()) throw new axios.CanceledError("Session cha
90 > nged");
91     return response;
92   },
93   async (error) => {
94     const originalRequest = error.config;
95 +   if (!originalRequest || (originalRequest._sessionCaptured && originalRequest._authSession !== tokenStorage.g
96 > etSessionId())) return Promise.reject(error);
97
98     // -----
99     // Only handle 401
100 @@ -152,7 +140,7 @@
101     * cleared authentication.
102     */
103 -   window.location.href = "/login";
104 +   // AuthContext reacts to auth-cleared; transient failures stay retryable.
105
106     return Promise.reject(refreshError);
107   }

```

```

1 import { createContext, useCallback, useContext, useEffect, useRef, useState } from "react";
2 import { login as loginApi, signup as signupApi, logout as logoutApi } from "../api/authApi";
3 import { getCurrentUser } from "../api/userApi";
4 import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
5 import { apiErrorMessage } from "../utils/apiError";
6
7 const AuthContext = createContext(null);
8 export const AuthProvider = ({ children }) => {
9   const [user, setUser] = useState(null);
10  const [loading, setLoading] = useState(true);
11  const [authError, setAuthError] = useState("");
12  const request = useRef(0);
13  const mounted = useRef(false);
14  const currentUser = useRef(null);
15  const replaceUser = useCallback(value => { currentUser.current = value; setUser(value); }, []);
16  const retryAuth = useCallback(async () => {
17    const version = ++request.current;
18    const session = tokenStorage.getSessionId();
19    const current = () => mounted.current && version === request.current && session === tokenStorage.getSessionId
20  > ());
21    setAuthError("");
22    if (!tokenStorage.getAccessToken()) { replaceUser(null); setLoading(false); return; }
23    if (!currentUser.current) setLoading(true);
24    try {
25      const account = await getCurrentUser();
26      if (!current() || !tokenStorage.getAccessToken()) return;
27      replaceUser(account);
28      tokenStorage.updateUser(account);
29    } catch (error) {
30      if (!current()) return;
31      replaceUser(null);
32      if ([401, 403].includes(error.response?.status)) tokenStorage.clear();
33      else setAuthError(apiErrorMessage(error, "Unable to verify your session. Please retry."));
34    } finally { if (current()) setLoading(false); }
35  }, [replaceUser]);
36
37  useEffect(() => {
38    mounted.current = true;
39    const cleared = () => { ++request.current; replaceUser(null); setAuthError(""); setLoading(false); };
40    let storageTimer;
41    const storageChanged = event => {
42      if (event.storageArea !== localStorage) return;
43      if (event.key === null || (event.key === "frndbook_access_token" && !tokenStorage.getAccessToken())) {
44        // Notify every local subscriber, including the live connections.
45        window.dispatchEvent(new Event(AUTH_CLEARED_EVENT));
46      } else if (["frndbook_session", "frndbook_user"].includes(event.key)) {
47        if (event.key === "frndbook_session") window.dispatchEvent(new Event(AUTH_CLEARED_EVENT));
48        clearTimeout(storageTimer);
49        storageTimer = setTimeout(() => {
50          if (tokenStorage.getAccessToken()) void retryAuth();
51        }, 0);
52      }
53    };
54    window.addEventListener(AUTH_CLEARED_EVENT, cleared);
55    window.addEventListener("storage", storageChanged);
56    // Bootstrap synchronizes external credentials with React state.
57    void retryAuth();
58    const invalidate = () => { mounted.current = false; ++request.current; };
59    return () => {
60      invalidate(); clearTimeout(storageTimer);
61      window.removeEventListener(AUTH_CLEARED_EVENT, cleared);
62      window.removeEventListener("storage", storageChanged);
63    };
64  }, [replaceUser, retryAuth]);
65
66  const login = async (email, password) => {
67    const version = ++request.current;
68    const response = await loginApi({ email, password });
69    if (!mounted.current || version !== request.current) throw new Error("Login was superseded");
70    tokenStorage.saveAuth(response.accessToken, response.refreshToken, response.user);

```

```
70     replaceUser(response.user); setAuthError(""); setLoading(false);
71     return response;
72   };
73   const signup = (name, email, password) => signupApi({ name, email, password });
74   const logout = async () => {
75     const session = tokenStorage.getSessionId();
76     ++request.current;
77     try { await logoutApi(); }
78     catch { /* Local logout still completes when the request fails. */ }
79     finally { if (session === tokenStorage.getSessionId()) tokenStorage.clear(); }
80   };
81   const updateUser = useCallback((account, expectedSession) => {
82     if (!mounted.current || !tokenStorage.getAccessToken() || expectedSession !== tokenStorage.getSessionId() ||
83       String(account?.id) !== String(currentUser.current?.id)) return false;
84     ++request.current; // An older /me response must not undo a saved profile.
85     replaceUser(account); tokenStorage.updateUser(account);
86     return true;
87   }, [replaceUser]);
88
89   return <AuthContext.Provider value={{ user, loading, authError, retryAuth,
90     sessionKey: tokenStorage.getSessionId(), isAuthenticated: !!user, login, signup, logout, updateUser }}>
91     {children}
92   </AuthContext.Provider>;
93 };
94 export const useAuth = () => {
95   const context = useContext(AuthContext);
96   if (!context) throw new Error("useAuth must be used inside AuthProvider");
97   return context;
98 };
```

```

1  --- before/src/auth/AuthContext.jsx
2  +++ after/src/auth/AuthContext.jsx
3  @@ -1,156 +1,98 @@
4  -import { createContext, useContext, useEffect, useState } from "react";
5  -
6  -import {
7  -  login as loginApi,
8  -  signup as signupApi,
9  -  logout as logoutApi,
10 -} from "../api/authApi";
11 -
12 +import { createContext, useCallbck, useContext, useEffect, useRef, useState } from "react";
13 +import { login as loginApi, signup as signupApi, logout as logoutApi } from "../api/authApi";
14 +import { getCurrentUser } from "../api/userApi";
15 -
16 -import { tokenStorage } from "../utils/tokenStorage";
17 +import { tokenStorage, AUTH_CLEARED_EVENT } from "../utils/tokenStorage";
18 +import { apiErrorMessage } from "../utils/apiError";
19
20 const AuthContext = createContext(null);
21 -
22 export const AuthProvider = ({ children }) => {
23   const [user, setUser] = useState(null);
24 -
25   const [loading, setLoading] = useState(true);
26 -
27 -  // =====
28 -  // INITIALIZE AUTHENTICATION
29 -  // =====
30 + const [authError, setAuthError] = useState("");
31 + const request = useRef(0);
32 + const mounted = useRef(false);
33 + const currentUser = useRef(null);
34 + const replaceUser = useCallback(value => { currentUser.current = value; setUser(value); }, []);
35 + const retryAuth = useCallback(async () => {
36 +   const version = ++request.current;
37 +   const session = tokenStorage.getSessionId();
38 +   const current = () => mounted.current && version === request.current && session === tokenStorage.getSessionI
39 +   > d();
40 +   setAuthError("");
41 +   if (!tokenStorage.getAccessToken()) { replaceUser(null); setLoading(false); return; }
42 +   if (!currentUser.current) setLoading(true);
43 +   try {
44 +     const account = await getCurrentUser();
45 +     if (!current() || !tokenStorage.getAccessToken()) return;
46 +     replaceUser(account);
47 +     tokenStorage.updateUser(account);
48 +   } catch (error) {
49 +     if (!current()) return;
50 +     replaceUser(null);
51 +     if ([401, 403].includes(error.response?.status)) tokenStorage.clear();
52 +     else setAuthError(apiErrorMessage(error, "Unable to verify your session. Please retry."));
53 +   } finally { if (current()) setLoading(false); }
54 + }, [replaceUser]);
55
56   useEffect(() => {
57 -     const initializeAuth = async () => {
58 -       const accessToken = tokenStorage.getAccessToken();
59 -
60 -       // -----
61 -       // No token = not authenticated
62 -       // -----
63 -
64 -       if (!accessToken) {
65 -         setUser(null);
66 -
67 -         setLoading(false);
68 -
69 -         return;

```

```

70 -
71 -     try {
72 -         /*
73 -          * Backend is the source of truth.
74 -          *
75 -          * We do NOT simply trust
76 -          * frndbook_user from localStorage.
77 -          *
78 -          * If access token is expired,
79 -          * axiosClient automatically attempts
80 -          * refresh and retries this request.
81 -          */
82 -
83 -         const currentUser = await getCurrentUser();
84 -
85 -         setUser(currentUser);
86 -
87 -         tokenStorage.updateUser(currentUser);
88 -     } catch (error) {
89 -         console.error("Authentication initialization failed:", error);
90 -
91 -         tokenStorage.clear();
92 -
93 -         setUser(null);
94 -     } finally {
95 -         setLoading(false);
96 +     mounted.current = true;
97 +     const cleared = () => { ++request.current; replaceUser(null); setAuthError(""); setLoading(false); };
98 +     let storageTimer;
99 +     const storageChanged = event => {
100 +         if (event.storageArea !== localStorage) return;
101 +         if (event.key === null || (event.key === "frndbook_access_token" && !tokenStorage.getAccessToken())) {
102 +             // Notify every local subscriber, including the live connections.
103 +             window.dispatchEvent(new Event(AUTH_CLEARED_EVENT));
104 +         } else if (["frndbook_session", "frndbook_user"].includes(event.key)) {
105 +             if (event.key === "frndbook_session") window.dispatchEvent(new Event(AUTH_CLEARED_EVENT));
106 +             clearTimeout(storageTimer);
107 +             storageTimer = setTimeout(() => {
108 +                 if (tokenStorage.getAccessToken()) void retryAuth();
109 +             }, 0);
110         }
111     };
112 -
113 -     initializeAuth();
114 - }, []);
115 -
116 - // =====
117 - // LOGIN
118 - // =====
119 + window.addEventListener(AUTH_CLEARED_EVENT, cleared);
120 + window.addEventListener("storage", storageChanged);
121 + // Bootstrap synchronizes external credentials with React state.
122 + void retryAuth();
123 + const invalidate = () => { mounted.current = false; ++request.current; };
124 + return () => {
125 +     invalidate(); clearTimeout(storageTimer);
126 +     window.removeEventListener(AUTH_CLEARED_EVENT, cleared);
127 +     window.removeEventListener("storage", storageChanged);
128 + };
129 + }, [replaceUser, retryAuth]);
130
131 const login = async (email, password) => {
132 -     const response = await loginApi({
133 -         email,
134 -         password,
135 -     });
136 -
137 -     tokenStorage.saveAuth(
138 -         response.accessToken,
139 -         response.refreshToken,

```

```

140 -     response.user,
141 - );
142 -
143 -     setUser(response.user);
144 -
145 +     const version = ++request.current;
146 +     const response = await loginApi({ email, password });
147 +     if (!mounted.current || version !== request.current) throw new Error("Login was superseded");
148 +     tokenStorage.saveAuth(response.accessToken, response.refreshToken, response.user);
149 +     replaceUser(response.user); setAuthError(""); setLoading(false);
150     return response;
151 };
152 + const signup = (name, email, password) => signupApi({ name, email, password });
153 + const logout = async () => {
154 +     const session = tokenStorage.getSessionId();
155 +     ++request.current;
156 +     try { await logoutApi(); }
157 +     catch { /* Local logout still completes when the request fails. */ }
158 +     finally { if (session === tokenStorage.getSessionId()) tokenStorage.clear(); }
159 + };
160 + const updateUser = useCallback((account, expectedSession) => {
161 +     if (!mounted.current || !tokenStorage.getAccessToken() || expectedSession !== tokenStorage.getSessionId() ||
162 +         String(account?.id) !== String(currentUser.current?.id)) return false;
163 +     ++request.current; // An older /me response must not undo a saved profile.
164 +     replaceUser(account); tokenStorage.updateUser(account);
165 +     return true;
166 + }, [replaceUser]);
167
168 - // =====
169 - // SIGNUP
170 - // =====
171 -
172 - const signup = async (name, email, password) => {
173 -     const response = await signupApi({
174 -         name,
175 -         email,
176 -         password,
177 -     });
178 -
179 -     return response;
180 - };
181 -
182 - // =====
183 - // LOGOUT
184 - // =====
185 -
186 - const logout = async () => {
187 -     try {
188 -         await logoutApi();
189 -     } catch (error) {
190 -         /*
191 -          * Even if the backend logout request
192 -          * fails, remove local authentication.
193 -          */
194 -
195 -         console.error("Logout API failed:", error);
196 -     } finally {
197 -         tokenStorage.clear();
198 -
199 -         setUser(null);
200 -     }
201 - };
202 -
203 - // =====
204 - // CONTEXT VALUE
205 - // =====
206 -
207 - const value = {
208 -     user,
209 -

```

```
210 -   loading,
211 -
212 -   isAuthenticated: !!user,
213 -
214 -   login,
215 -
216 -   signup,
217 -
218 -   logout,
219 - };
220 -
221 - return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
222 + return <AuthContext.Provider value={{ user, loading, authError, retryAuth,
223 +   sessionKey: tokenStorage.getSessionId(), isAuthenticated: !!user, login, signup, logout, updateUser }}>
224 +   {children}
225 + </AuthContext.Provider>;
226 };
227 -
228 export const useAuth = () => {
229   const context = useContext(AuthContext);
230 -
231 -   if (!context) {
232 -     throw new Error("useAuth must be used inside AuthProvider");
233 -   }
234 -
235 +   if (!context) throw new Error("useAuth must be used inside AuthProvider");
236   return context;
237 };
```

src/auth/ProtectedRoute.jsx

FULL CURRENT CONTENT

```
1 import { Navigate } from "react-router-dom";
2
3 import { useAuth } from "../AuthContext";
4
5 const ProtectedRoute = ({ children }) => {
6   const { user, loading, authError, retryAuth } = useAuth();
7
8   if (loading) {
9     return <div role="status" className="loading-screen">Loading...</div>;
10  }
11
12  if (authError) {
13    return <main className="loading-screen"><p role="alert">{authError}</p>
14      <button type="button" onClick={retryAuth}>Retry session check</button></main>;
15  }
16
17  if (!user) {
18    return <Navigate to="/login" replace />;
19  }
20
21  return children;
22 };
23
24 export default ProtectedRoute;
```

src/auth/ProtectedRoute.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/auth/ProtectedRoute.jsx
2  +++ after/src/auth/ProtectedRoute.jsx
3  @@ -3,10 +3,15 @@
4   import { useAuth } from "../AuthContext";
5
6   const ProtectedRoute = ({ children }) => {
7   -   const { user, loading } = useAuth();
8   +   const { user, loading, authError, retryAuth } = useAuth();
9
10    if (loading) {
11      return <div role="status" className="loading-screen">Loading...</div>;
12  + }
13  +
14  + if (authError) {
15  +   return <main className="loading-screen"><p role="alert">{authError}</p>
16  +     <button type="button" onClick={retryAuth}>Retry session check</button></main>;
17  + }
18
19   if (!user) {
```

```
1 import { Navigate } from "react-router-dom";
2
3 import { useAuth } from "../AuthContext";
4
5 const PublicRoute = ({ children }) => {
6   const { user, loading, authError, retryAuth } = useAuth();
7
8   if (loading) {
9     return <div role="status" className="loading-screen">Loading...</div>;
10  }
11
12  if (authError) {
13    return <main className="loading-screen"><p role="alert">{authError}</p>
14      <button type="button" onClick={retryAuth}>Retry session check</button></main>;
15  }
16
17  /*
18   * User is already authenticated.
19   *
20   * Therefore:
21   *
22   * /login → /
23   * /signup → /
24   */
25
26  if (user) {
27    return <Navigate to="/" replace />;
28  }
29
30  return children;
31 };
32
33 export default PublicRoute;
```

src/auth/PublicRoute.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/auth/PublicRoute.jsx
2  +++ after/src/auth/PublicRoute.jsx
3  @@ -3,10 +3,15 @@
4   import { useAuth } from "../AuthContext";
5
6   const PublicRoute = ({ children }) => {
7   -   const { user, loading } = useAuth();
8   +   const { user, loading, authError, retryAuth } = useAuth();
9
10    if (loading) {
11      return <div role="status" className="loading-screen">Loading...</div>;
12  + }
13  +
14  + if (authError) {
15  +   return <main className="loading-screen"><p role="alert">{authError}</p>
16  +     <button type="button" onClick={retryAuth}>Retry session check</button></main>;
17  + }
18
19  /*
```

```

1  import { apiErrorMessage } from "../../utils/apiError.js";
2  import usersStyles from "../../styles/users.module.css";
3  import { bindStyles } from "../../utils/bindStyles";
4  import StatusMessage from "../../ui/StatusMessage";
5  import { useEffect, useState } from "react";
6
7  import {
8    addRecentSearch,
9    clearRecentSearches,
10   getRecentSearches,
11   searchUsers,
12 } from "../../api/userApi";
13
14 import { useAuth } from "../../auth/AuthContext";
15
16 import UserCard from "../UserCard";
17
18 const css = bindStyles(usersStyles);
19
20 const UserSearch = () => {
21   const { user: currentUser } = useAuth();
22
23   const [searchTerm, setSearchTerm] = useState("");
24
25   const [results, setResults] = useState([]);
26
27   const [recentSearches, setRecentSearches] = useState([]);
28
29   const [page, setPage] = useState(0);
30
31   const [totalPages, setTotalPages] = useState(0);
32
33   const [loading, setLoading] = useState(false);
34
35   const [recentLoading, setRecentLoading] = useState(true);
36
37   const [error, setError] = useState("");
38
39   // =====
40   // LOAD RECENT SEARCHES
41   // =====
42
43   useEffect(() => {
44     const loadRecentSearches = async () => {
45       try {
46         setRecentLoading(true);
47
48         const data = await getRecentSearches();
49
50         setRecentSearches(data || []);
51       } catch (error) {
52         console.error("Failed to load recent searches:", error);
53       } finally {
54         setRecentLoading(false);
55       }
56     };
57
58     loadRecentSearches();
59   }, []);
60
61   // =====
62   // SEARCH
63   // =====
64
65   useEffect(() => {
66     const trimmedSearch = searchTerm.trim();
67
68     if (!trimmedSearch) {
69       setResults([]);
70     }

```

```

71     setPage(0);
72
73     setTotalPages(0);
74
75     setError("");
76
77     return;
78 }
79
80 const timer = setTimeout(async () => {
81   try {
82     setLoading(true);
83
84     setError("");
85
86     const data = await searchUsers(trimmedSearch, 0, 10);
87
88     const users = data?.content || [];
89
90     /*
91      * Don't show the currently logged-in
92      * user in user search.
93      */
94
95     const filteredUsers = users.filter(
96       (searchedUser) => searchedUser.id !== currentUser?.id,
97     );
98
99     setResults(filteredUsers);
100
101     setPage(data?.number ?? 0);
102
103     setTotalPages(data?.totalPages ?? 0);
104   } catch (error) {
105     console.error("User search failed:", error);
106
107     setError(apiErrorMessage(error, "Unable to search users"));
108
109     setResults([]);
110   } finally {
111     setLoading(false);
112   }
113 }, 400);
114
115 return () => {
116   clearTimeout(timer);
117 };
118 }, [searchTerm, currentUser?.id]);
119
120 // =====
121 // LOAD PAGE
122 // =====
123
124 const loadPage = async (nextPage) => {
125   if (!searchTerm.trim()) {
126     return;
127   }
128
129   try {
130     setLoading(true);
131
132     setError("");
133
134     const data = await searchUsers(searchTerm.trim(), nextPage, 10);
135
136     const users = data?.content || [];
137
138     const filteredUsers = users.filter(
139       (searchedUser) => searchedUser.id !== currentUser?.id,
140     );

```

```

141
142     setResults(filteredUsers);
143
144     setPage(data?.number ?? nextPage);
145
146     setTotalPages(data?.totalPages ?? 0);
147   } catch (error) {
148     console.error("Failed to load search page:", error);
149
150     setError(apiErrorMessage(error, "Unable to load results"));
151   } finally {
152     setLoading(false);
153   }
154 };
155
156 // =====
157 // USER CLICK
158 // =====
159
160 const handleUserClick = async (selectedUser) => {
161   try {
162     await addRecentSearch(selectedUser.id);
163
164     /*
165      * Refresh recent searches so the UI
166      * immediately reflects the backend state.
167      */
168
169     const updatedRecentSearches = await getRecentSearches();
170
171     setRecentSearches(updatedRecentSearches || []);
172   } catch (error) {
173     console.error("Failed to save recent search:", error);
174   }
175 };
176
177 // =====
178 // CLEAR RECENT SEARCHES
179 // =====
180
181 const handleClearRecentSearches = async () => {
182   try {
183     await clearRecentSearches();
184
185     setRecentSearches([]);
186   } catch (error) {
187     console.error("Failed to clear recent searches:", error);
188   }
189 };
190
191 return (
192   <section className={css("user-search")}>
193     <div className={css("section-header")}>
194       <h2>Find People</h2>
195     </div>
196
197     <input
198       type="text"
199       className={css("search-input")}
200       aria-label="Search people by name"
201       placeholder="Search users by name..."
202       value={searchTerm}
203       onChange={(event) => setSearchTerm(event.target.value)}
204     />
205
206     {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
207
208     {loading && <p className={css("search-status")}>Searching...</p>}
209
210     {!loading && searchTerm.trim() && results.length === 0 && !error && (

```

```

211     <p className={css("search-status")}>No users found.</p>
212   )}
213
214   <div className={css("user-results")}>
215     {results.map((searchedUser) => (
216       <UserCard
217         key={searchedUser.id}
218         user={searchedUser}
219         onClick={handleUserClick}
220       />
221     ))}
222   </div>
223
224   {totalPages > 1 && searchTerm.trim() && (
225     <div className={css("pagination")}>
226       <button
227         disabled={loading || page === 0}
228         onClick={() => loadPage(page - 1)}
229       >
230         Previous
231       </button>
232
233       <span>
234         Page {page + 1} of {totalPages}
235       </span>
236
237       <button
238         disabled={loading || page >= totalPages - 1}
239         onClick={() => loadPage(page + 1)}
240       >
241         Next
242       </button>
243     </div>
244   )}
245
246   {!searchTerm.trim() && (
247     <section className={css("recent-searches")}>
248       <div className={css("section-header")}>
249         <h3>Recent Searches</h3>
250
251         {recentSearches.length > 0 && (
252           <button
253             className={css("text-button")}
254             onClick={handleClearRecentSearches}
255           >
256             Clear
257           </button>
258         )}
259       </div>
260
261       {recentLoading && (
262         <p className={css("search-status")}>Loading recent searches...</p>
263       )}
264
265       {!recentLoading && recentSearches.length === 0 && (
266         <p className={css("search-status")}>No recent searches.</p>
267       )}
268
269       <div className={css("user-results")}>
270         {recentSearches.map((recentUser) => (
271           <UserCard
272             key={recentUser.id}
273             user={recentUser}
274             onClick={handleUserClick}
275           />
276         ))}
277       </div>
278     </section>
279   )}
280 </section>

```

src/components/users/UserSearch.jsx

FULL CURRENT CONTENT

```
281   );  
282   };  
283  
284   export default UserSearch;
```

src/components/users/UserSearch.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/components/users/UserSearch.jsx
2  +++ after/src/components/users/UserSearch.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../../utils/apiError.js";
5  import usersStyles from "../../styles/users.module.css";
6  import { bindStyles } from "../../utils/bindStyles";
7  import StatusMessage from "../ui/StatusMessage";
8  @@ -103,7 +104,7 @@
9      } catch (error) {
10         console.error("User search failed:", error);
11
12 -         setError(error.response?.data?.message || "Unable to search users");
13 +         setError(apiErrorMessage(error, "Unable to search users"));
14
15         setResults([]);
16     } finally {
17 @@ -146,7 +147,7 @@
18     } catch (error) {
19         console.error("Failed to load search page:", error);
20
21 -         setError(error.response?.data?.message || "Unable to load results");
22 +         setError(apiErrorMessage(error, "Unable to load results"));
23     } finally {
24         setLoading(false);
25     }
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import { createContext, useContext, useEffect, useRef, useState } from "react";
3
4  import { useAuth } from "../auth/AuthContext";
5
6  import {
7    getNotifications,
8    getUnreadNotificationCount,
9    markNotificationAsRead,
10   markAllNotificationsAsRead,
11 } from "../api/notificationApi";
12
13 import { createNotificationWebSocket } from "../services/webSocketService";
14
15 const NotificationContext = createContext(null);
16
17 const NotificationState = ({ children }) => {
18   const { user } = useAuth();
19
20   const [notifications, setNotifications] = useState([]);
21   const [unreadCount, setUnreadCount] = useState(0);
22
23   const [loading, setLoading] = useState(false);
24   const [error, setError] = useState("");
25
26   const websocketRef = useRef(null);
27
28   /*
29    * Keep track of notification IDs already processed
30    * by this client.
31    */
32   const notificationIdsRef = useRef(new Set());
33
34   // =====
35   // RESET WHEN LOGGED OUT
36   // =====
37
38   useEffect(() => {
39     if (user) {
40       return;
41     }
42
43     setNotifications([]);
44     setUnreadCount(0);
45     setError("");
46     setLoading(false);
47
48     notificationIdsRef.current.clear();
49   }, [user]);
50
51   // =====
52   // LOAD INITIAL UNREAD COUNT
53   // =====
54
55   useEffect(() => {
56     if (!user) {
57       return;
58     }
59
60     let cancelled = false;
61
62     const loadInitialUnreadCount = async () => {
63       try {
64         setError("");
65
66         const count = await getUnreadNotificationCount();
67
68         if (!cancelled) {
69           setUnreadCount(Number(count) || 0);
70         }
71       }

```

```

71     } catch (error) {
72       console.error("Failed to load notification count:", error);
73     }
74     if (!cancelled) {
75       setUnreadCount(0);
76     }
77   }
78   };
79
80   loadInitialUnreadCount();
81
82   return () => {
83     cancelled = true;
84   };
85 }, [user?.id]);
86
87 // =====
88 // WEBSOCKET
89 // =====
90
91 useEffect(() => {
92   if (!user) {
93     return;
94   }
95
96   let cancelled = false;
97
98   const handleNotification = (notification) => {
99     if (cancelled || !notification?.id) {
100       return;
101     }
102
103     const notificationId = String(notification.id);
104
105     /*
106      * Ignore duplicate WebSocket deliveries.
107      */
108     if (notificationIdsRef.current.has(notificationId)) {
109       return;
110     }
111
112     /*
113      * Mark the notification as processed BEFORE
114      * updating React state.
115      */
116     notificationIdsRef.current.add(notificationId);
117
118     setNotifications((currentNotifications) => {
119       /*
120        * Extra protection against duplicates already
121        * present in the current state.
122        */
123       const alreadyExists = currentNotifications.some(
124         (item) => String(item.id) === notificationId,
125       );
126
127       if (alreadyExists) {
128         return currentNotifications;
129       }
130
131       return [notification, ...currentNotifications];
132     });
133
134     /*
135      * The notification is known to be new at this
136      * point, so updating the unread count is safe.
137      */
138     if (!notification.read) {
139       setUnreadCount((currentCount) => currentCount + 1);
140     }

```

```

141     };
142
143     const handleConnect = () => {
144         if (cancelled) {
145             return;
146         }
147     };
148
149     const handleDisconnect = () => {
150         if (cancelled) {
151             return;
152         }
153     };
154
155     const handleError = (error) => {
156         if (cancelled) {
157             return;
158         }
159
160         console.error("Notification WebSocket error:", error);
161     };
162
163     const websocket = createNotificationWebSocket({
164         onNotification: handleNotification,
165         onConnect: handleConnect,
166         onDisconnect: handleDisconnect,
167         onError: handleError,
168     });
169
170     if (!websocket) {
171         return;
172     }
173
174     websocketRef.current = websocket;
175
176     websocket.connect();
177
178     return () => {
179         cancelled = true;
180
181         const currentWebSocket = websocketRef.current;
182
183         websocketRef.current = null;
184
185         if (currentWebSocket) {
186             currentWebSocket.disconnect().catch((error) => {
187                 console.error("Failed to disconnect notification WebSocket:", error);
188             });
189         }
190     };
191 }, [user?.id]);
192
193 // =====
194 // LOAD NOTIFICATIONS
195 // =====
196
197 const loadNotifications = async (page = 0, size = 10) => {
198     try {
199         setLoading(true);
200         setError("");
201
202         const data = await getNotifications(page, size);
203
204         const loadedNotifications = data?.content || [];
205
206         /*
207          * Keep the local ID set synchronized with
208          * notifications loaded from the backend.
209          */
210         notificationIdsRef.current = new Set(

```

```

211     loadedNotifications
212     .filter((notification) => notification?.id)
213     .map((notification) => String(notification.id)),
214 );
215
216     setNotifications(loadedNotifications);
217
218     return data;
219 } catch (error) {
220     console.error("Failed to load notifications:", error);
221
222     setError(apiErrorMessage(error, "Unable to load notifications"));
223
224     throw error;
225 } finally {
226     setLoading(false);
227 }
228 };
229
230 // =====
231 // REFRESH UNREAD COUNT
232 // =====
233
234 const refreshUnreadCount = async () => {
235     try {
236         const count = await getUnreadNotificationCount();
237
238         setUnreadCount(Number(count) || 0);
239
240         return Number(count) || 0;
241     } catch (error) {
242         console.error("Failed to refresh notification count:", error);
243
244         throw error;
245     }
246 };
247
248 // =====
249 // MARK ONE AS READ
250 // =====
251
252 const markAsRead = async (notificationId) => {
253     if (!notificationId) {
254         return;
255     }
256
257     const notification = notifications.find(
258         (item) => item.id === notificationId,
259     );
260
261     if (notification?.read) {
262         return;
263     }
264
265     try {
266         await markNotificationAsRead(notificationId);
267
268         setNotifications((currentNotifications) =>
269             currentNotifications.map((item) =>
270                 item.id === notificationId
271                 ? {
272                     ...item,
273                     read: true,
274                 }
275                 : item,
276             ),
277         );
278
279         setUnreadCount((currentCount) => Math.max(0, currentCount - 1));
280     } catch (error) {

```

```

281     console.error("Failed to mark notification as read:", error);
282
283     throw error;
284   }
285 };
286
287 // =====
288 // MARK ALL AS READ
289 // =====
290
291 const markAllAsRead = async () => {
292   try {
293     await markAllNotificationsAsRead();
294
295     setNotifications((currentNotifications) =>
296       currentNotifications.map((notification) => ({
297         ...notification,
298         read: true,
299       })),
300     );
301
302     setUnreadCount(0);
303   } catch (error) {
304     console.error("Failed to mark all notifications as read:", error);
305
306     throw error;
307   }
308 };
309
310 // =====
311 // CONTEXT VALUE
312 // =====
313
314 const value = {
315   notifications,
316   unreadCount,
317
318   loading,
319   error,
320
321   loadNotifications,
322   refreshUnreadCount,
323
324   markAsRead,
325   markAllAsRead,
326 };
327
328 return (
329   <NotificationContext.Provider value={value}>
330     {children}
331   </NotificationContext.Provider>
332 );
333 };
334
335 // =====
336 // HOOK
337 // =====
338
339 // Account changes dispose all notification requests/state with their provider lifetime.
340 export const NotificationProvider = ({ children }) => {
341   const { user, sessionKey } = useAuth();
342   return <NotificationState key={` ${sessionKey}:${user?.id}`}>{children}</NotificationState>;
343 };
344
345 export const useNotifications = () => {
346   const context = useContext(NotificationContext);
347
348   if (!context) {
349     throw new Error(
350       "useNotifications must be used inside NotificationProvider",

```

src/context/NotificationContext.jsx

FULL CURRENT CONTENT

```
351     );  
352   }  
353  
354   return context;  
355 };
```

src/context/NotificationContext.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/context/NotificationContext.jsx
2  +++ after/src/context/NotificationContext.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../utils/apiError.js";
5  import { createContext, useContext, useEffect, useRef, useState } from "react";
6
7  import { useAuth } from "../auth/AuthContext";
8  @@ -13,7 +14,7 @@
9
10 const NotificationContext = createContext(null);
11
12 -export const NotificationProvider = ({ children }) => {
13 +const NotificationState = ({ children }) => {
14     const { user } = useAuth();
15
16     const [notifications, setNotifications] = useState([]);
17  @@ -218,7 +219,7 @@
18     } catch (error) {
19         console.error("Failed to load notifications:", error);
20
21 -        setError(error.response?.data?.message || "Unable to load notifications");
22 +        setError(apiErrorMessage(error, "Unable to load notifications"));
23
24         throw error;
25     } finally {
26  @@ -335,6 +336,12 @@
27     // HOOK
28     // =====
29
30 +// Account changes dispose all notification requests/state with their provider lifetime.
31 +export const NotificationProvider = ({ children }) => {
32 +    const { user, sessionKey } = useAuth();
33 +    return <NotificationState key={`${sessionKey}:${user?.id}`}>{children}</NotificationState>;
34 +};
35 +
36 export const useNotifications = () => {
37     const context = useContext(NotificationContext);
38
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import authStyles from "../styles/auth.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import Button from "../components/ui/Button";
8  import { useState } from "react";
9  import { Link } from "react-router-dom";
10
11 import { forgotPassword } from "../api/authApi";
12
13 const css = bindStyles(authStyles);
14
15 const ForgotPassword = () => {
16   const [email, setEmail] = useState("");
17
18   const [error, setError] = useState("");
19   const [fieldErrors, setFieldErrors] = useState({});
20   const [success, setSuccess] = useState("");
21
22   const [loading, setLoading] = useState(false);
23
24   const handleSubmit = async (event) => {
25     event.preventDefault();
26
27     setError("");
28     setFieldErrors({});
29     setSuccess("");
30     setLoading(true);
31
32     try {
33       await forgotPassword(email);
34
35       /*
36        * Always show the same message.
37        *
38        * This prevents revealing whether
39        * the email exists in FrndBook.
40        */
41       setSuccess(
42         "If an account exists for this email, a password reset link has been sent.",
43       );
44     } catch (error) {
45       setFieldErrors(apiError(error).fieldErrors);
46       console.error(error);
47
48       setError(
49         apiErrorMessage(error, "Unable to process your request"),
50       );
51     } finally {
52       setLoading(false);
53     }
54   };
55
56   return (
57     <div className={css("auth-page")}>
58       <div className={css("auth-card")}>
59         <h1>FrndBook</h1>
60
61         <h2>Forgot Password</h2>
62
63         <p>
64           Enter your email and we'll send you a password reset link if an
65           account exists.
66         </p>
67
68         <form onSubmit={handleSubmit}>
69           <FormField label="Email" error={fieldErrors.email} autoComplete="email"
70             type="email"

```

src/pages/ForgotPassword.jsx

FULL CURRENT CONTENT

```
71         placeholder="Email"
72         value={email}
73         onChange={(event) => setEmail(event.target.value)}
74         required
75     />
76
77     {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
78
79     {success && <StatusMessage tone="success" className={css("success")}>{success}</StatusMessage>}
80
81     <Button type="submit" disabled={loading}>
82       {loading ? "Sending..." : "Send Reset Link"}
83     </Button>
84 </form>
85
86 <p>
87   <Link to="/login">Back to Login</Link>
88 </p>
89 </div>
90 </div>
91 );
92 };
93
94 export default ForgotPassword;
```

src/pages/ForgotPassword.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/ForgotPassword.jsx
2  +++ after/src/pages/ForgotPassword.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import authStyles from "../styles/auth.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -14,6 +16,7 @@
10  const [email, setEmail] = useState("");
11
12  const [error, setError] = useState("");
13  + const [fieldErrors, setFieldErrors] = useState({});
14  const [success, setSuccess] = useState("");
15
16  const [loading, setLoading] = useState(false);
17  @@ -22,6 +25,7 @@
18  event.preventDefault();
19
20  setError("");
21  + setFieldErrors({});
22  setSuccess("");
23  setLoading(true);
24
25  @@ -38,10 +42,11 @@
26  "If an account exists for this email, a password reset link has been sent.",
27  );
28  } catch (error) {
29  + setFieldErrors(apiError(error).fieldErrors);
30  console.error(error);
31
32  setError(
33  - error.response?.data?.message || "Unable to process your request",
34  + apiErrorMessage(error, "Unable to process your request"),
35  );
36  } finally {
37  setLoading(false);
38  @@ -61,7 +66,7 @@
39  </p>
40
41  <form onSubmit={handleSubmit}>
42  - <FormField label="Email" autoComplete="email"
43  + <FormField label="Email" error={fieldErrors.email} autoComplete="email"
44  type="email"
45  placeholder="Email"
46  value={email}
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import friendsStyles from "../styles/friends.module.css";
3  import { bindStyles } from "../utils/bindStyles";
4  import StatusMessage from "../components/ui/StatusMessage";
5  import PageContainer from "../components/ui/PageContainer";
6  import Card from "../components/ui/Card";
7  import { useEffect, useState } from "react";
8
9  import Navbar from "../components/layout/Navbar";
10
11 import FriendRequestCard from "../components/users/FriendRequestCard";
12
13 import {
14   getReceivedFriendRequests,
15   getSentFriendRequests,
16   acceptFriendRequest,
17   rejectFriendRequest,
18 } from "../api/friendApi";
19
20 const css = bindStyles(friendsStyles);
21
22 const FriendRequests = () => {
23   const [receivedRequests, setReceivedRequests] = useState([]);
24
25   const [sentRequests, setSentRequests] = useState([]);
26
27   const [loading, setLoading] = useState(true);
28
29   const [error, setError] = useState("");
30
31   const [actionRequestId, setActionRequestId] = useState(null);
32
33   // =====
34   // LOAD REQUESTS
35   // =====
36
37   const loadRequests = async () => {
38     try {
39       setLoading(true);
40       setError("");
41
42       const [received, sent] = await Promise.all([
43         getReceivedFriendRequests(),
44         getSentFriendRequests(),
45       ]);
46
47       setReceivedRequests(received || []);
48       setSentRequests(sent || []);
49     } catch (error) {
50       console.error("Failed to load friend requests:", error);
51
52       setError(
53         apiErrorMessage(error, "Unable to load friend requests"),
54       );
55     } finally {
56       setLoading(false);
57     }
58   };
59
60   useEffect(() => {
61     loadRequests();
62   }, []);
63
64   // =====
65   // ACCEPT
66   // =====
67
68   const handleAccept = async (requestId) => {
69     try {
70       setError("");

```

```

71     setActionRequestId(requestId);
72
73     await acceptFriendRequest(requestId);
74
75     setReceivedRequests((requests) =>
76       requests.filter((request) => request.id !== requestId),
77     );
78   } catch (error) {
79     console.error("Failed to accept friend request:", error);
80
81     setError(
82       apiErrorMessage(error, "Unable to accept friend request"),
83     );
84   } finally {
85     setActionRequestId(null);
86   }
87 };
88
89 // =====
90 // REJECT
91 // =====
92
93 const handleReject = async (requestId) => {
94   try {
95     setError("");
96     setActionRequestId(requestId);
97
98     await rejectFriendRequest(requestId);
99
100    setReceivedRequests((requests) =>
101      requests.filter((request) => request.id !== requestId),
102    );
103   } catch (error) {
104     console.error("Failed to reject friend request:", error);
105
106     setError(
107       apiErrorMessage(error, "Unable to reject friend request"),
108     );
109   } finally {
110     setActionRequestId(null);
111   }
112 };
113
114 // =====
115 // LOADING
116 // =====
117
118 if (loading) {
119   return (
120     <>
121       <Navbar />
122
123       <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading friend requ
124     </div>
125   );
126 }
127
128 // =====
129 // UI
130 // =====
131
132 return (
133   <>
134     <Navbar />
135
136     <PageContainer className={css("friend-requests-page")}>
137       <Card className={css("friend-requests-card")}>
138         <div className={css("friends-header")}>
139           <div>

```

```

140         <h1>Friend Requests</h1>
141
142         <p>Manage your incoming and outgoing requests.</p>
143     </div>
144 </div>
145
146 {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
147
148 {/* =====
149     RECEIVED
150     ===== */}
151
152 <section className={css("request-section")}>
153     <div className={css("request-section-header")}>
154         <h2>Received</h2>
155
156         <span>{receivedRequests.length}</span>
157     </div>
158
159     {receivedRequests.length === 0 ? (
160         <p className={css("request-empty")}>No pending friend requests.</p>
161     ) : (
162         <div className={css("friend-request-list")}>
163             {receivedRequests.map((request) => (
164                 <FriendRequestCard
165                     key={request.id}
166                     request={request}
167                     type="received"
168                     onAccept={handleAccept}
169                     onReject={handleReject}
170                     actionLoading={actionRequestId === request.id}
171                 />
172             ))}
173         </div>
174     )}
175 </section>
176
177 {/* =====
178     SENT
179     ===== */}
180
181 <section className={css("request-section")}>
182     <div className={css("request-section-header")}>
183         <h2>Sent</h2>
184
185         <span>{sentRequests.length}</span>
186     </div>
187
188     {sentRequests.length === 0 ? (
189         <p className={css("request-empty")}>No pending sent requests.</p>
190     ) : (
191         <div className={css("friend-request-list")}>
192             {sentRequests.map((request) => (
193                 <FriendRequestCard
194                     key={request.id}
195                     request={request}
196                     type="sent"
197                 />
198             ))}
199         </div>
200     )}
201 </section>
202 </Card>
203 </PageContainer>
204 </>
205 );
206 };
207
208 export default FriendRequests;

```

src/pages/FriendRequests.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/FriendRequests.jsx
2  +++ after/src/pages/FriendRequests.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../utils/apiError.js";
5  import friendsStyles from "../styles/friends.module.css";
6  import { bindStyles } from "../utils/bindStyles";
7  import StatusMessage from "../components/ui/StatusMessage";
8  @@ -49,7 +50,7 @@
9      console.error("Failed to load friend requests:", error);
10
11      setError(
12  -      error.response?.data?.message || "Unable to load friend requests",
13  +      apiErrorMessage(error, "Unable to load friend requests"),
14  );
15  } finally {
16      setLoading(false);
17  @@ -78,7 +79,7 @@
18      console.error("Failed to accept friend request:", error);
19
20      setError(
21  -      error.response?.data?.message || "Unable to accept friend request",
22  +      apiErrorMessage(error, "Unable to accept friend request"),
23  );
24  } finally {
25      setActionRequestId(null);
26  @@ -103,7 +104,7 @@
27      console.error("Failed to reject friend request:", error);
28
29      setError(
30  -      error.response?.data?.message || "Unable to reject friend request",
31  +      apiErrorMessage(error, "Unable to reject friend request"),
32  );
33  } finally {
34      setActionRequestId(null);
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import friendsStyles from "../styles/friends.module.css";
3  import { bindStyles } from "../utils/bindStyles";
4  import StatusMessage from "../components/ui/StatusMessage";
5  import PageContainer from "../components/ui/PageContainer";
6  import Card from "../components/ui/Card";
7  import { useEffect, useState } from "react";
8
9  import Navbar from "../components/layout/Navbar";
10
11 import FriendCard from "../components/users/FriendCard";
12
13 import { getFriends, removeFriend } from "../api/friendApi";
14
15 const css = bindStyles(friendsStyles);
16
17 const Friends = () => {
18   const [friends, setFriends] = useState([]);
19
20   const [loading, setLoading] = useState(true);
21
22   const [error, setError] = useState("");
23
24   const [removingFriendId, setRemovingFriendId] = useState(null);
25
26   // =====
27   // LOAD FRIENDS
28   // =====
29
30   const loadFriends = async () => {
31     try {
32       setLoading(true);
33       setError("");
34
35       const data = await getFriends();
36
37       setFriends(data || []);
38     } catch (error) {
39       console.error("Failed to load friends:", error);
40
41       setError(apiErrorMessage(error, "Unable to load friends"));
42     } finally {
43       setLoading(false);
44     }
45   };
46
47   useEffect(() => {
48     loadFriends();
49   }, []);
50
51   // =====
52   // REMOVE FRIEND
53   // =====
54
55   const handleRemoveFriend = async (friend) => {
56     if (!friend) {
57       return;
58     }
59
60     const confirmed = window.confirm(
61       `Are you sure you want to remove ${friend.name} from your friends?`,
62     );
63
64     if (!confirmed) {
65       return;
66     }
67
68     try {
69       setError("");
70       setRemovingFriendId(friend.id);

```

```

71
72     await removeFriend(friend.id);
73
74     setFriends((currentFriends) =>
75         currentFriends.filter(
76             (currentFriend) => currentFriend.id !== friend.id,
77         ),
78     );
79 } catch (error) {
80     console.error("Failed to remove friend:", error);
81
82     setError(apiErrorMessage(error, "Unable to remove friend"));
83 } finally {
84     setRemovingFriendId(null);
85 }
86 };
87
88 // =====
89 // LOADING
90 // =====
91
92 if (loading) {
93     return (
94         <>
95             <Navbar />
96
97             <div id="main-content" tabIndex=-1 role="status" className={css("loading-screen")}>Loading friends...</
98 > div>
99         </>
100     );
101 }
102 // =====
103 // UI
104 // =====
105
106 return (
107     <>
108         <Navbar />
109
110         <PageContainer className={css("friends-page")}>
111             <Card className={css("friends-card")}>
112                 <div className={css("friends-header")}>
113                     <div>
114                         <h1>Friends</h1>
115
116                         <p>
117                             {friends.length} {friends.length === 1 ? "friend" : "friends"}
118                         </p>
119                     </div>
120                 </div>
121
122                 {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
123
124                 {!error && friends.length === 0 && (
125                     <div className={css("friends-empty")}>
126                         <h2>No friends yet</h2>
127
128                         <p>Search for people and send them a friend request.</p>
129                     </div>
130                 )}
131
132                 {friends.length > 0 && (
133                     <div className={css("friends-list")}>
134                         {friends.map((friend) => (
135                             <FriendCard
136                                 key={friend.id}
137                                 friend={friend}
138                                 onRemove={handleRemoveFriend}
139                                 removing={removingFriendId === friend.id}

```

src/pages/Friends.jsx

FULL CURRENT CONTENT

```
140             />
141         }}}
142     </div>
143   )}
144 </Card>
145 </PageContainer>
146 </>
147   );
148 };
149
150 export default Friends;
```

src/pages/Friends.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Friends.jsx
2  +++ after/src/pages/Friends.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../utils/apiError.js";
5  import friendsStyles from "../styles/friends.module.css";
6  import { bindStyles } from "../utils/bindStyles";
7  import StatusMessage from "../components/ui/StatusMessage";
8  @@ -37,7 +38,7 @@
9      } catch (error) {
10         console.error("Failed to load friends:", error);
11
12 -         setError(error.response?.data?.message || "Unable to load friends");
13 +         setError(apiErrorMessage(error, "Unable to load friends"));
14     } finally {
15         setLoading(false);
16     }
17  @@ -78,7 +79,7 @@
18     } catch (error) {
19         console.error("Failed to remove friend:", error);
20
21 -         setError(error.response?.data?.message || "Unable to remove friend");
22 +         setError(apiErrorMessage(error, "Unable to remove friend"));
23     } finally {
24         setRemovingFriendId(null);
25     }
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import authStyles from "../styles/auth.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import Button from "../components/ui/Button";
8  import { useState } from "react";
9  import { Link, useNavigate } from "react-router-dom";
10
11 import { useAuth } from "../auth/AuthContext";
12
13 const css = bindStyles(authStyles);
14
15 const Login = () => {
16   const navigate = useNavigate();
17
18   const { login } = useAuth();
19
20   const [email, setEmail] = useState("");
21
22   const [password, setPassword] = useState("");
23
24   const [error, setError] = useState("");
25   const [fieldErrors, setFieldErrors] = useState({});
26
27   const [loading, setLoading] = useState(false);
28
29   const handleSubmit = async (event) => {
30     event.preventDefault();
31
32     setError("");
33     setFieldErrors({});
34     setLoading(true);
35
36     try {
37       await login(email, password);
38
39       navigate("/");
40     } catch (error) {
41       setFieldErrors(apiError(error).fieldErrors);
42       console.error(error);
43
44       setError(apiErrorMessage(error, "Login failed"));
45     } finally {
46       setLoading(false);
47     }
48   };
49
50   return (
51     <div className={css("auth-page")}>
52       <div className={css("auth-card")}>
53         <h1>FrndBook</h1>
54
55         <h2>Login</h2>
56
57         <form onSubmit={handleSubmit}>
58           <FormField label="Email" error={fieldErrors.email} autoComplete="email"
59             type="email"
60             placeholder="Email"
61             value={email}
62             onChange={(event) => setEmail(event.target.value)}
63             required
64           />
65
66           <FormField label="Password" error={fieldErrors.password} autoComplete="current-password"
67             type="password"
68             placeholder="Password"
69             value={password}
70             onChange={(event) => setPassword(event.target.value)}

```

```
71         required
72     />
73
74     {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
75
76     <Button type="submit" disabled={loading}>
77       {loading ? "Logging in..." : "Login"}
78     </Button>
79 </form>
80
81 <p>
82   <Link to="/forgot-password">Forgot password?</Link>
83 </p>
84
85 <p>
86   Don't have an account? <Link to="/signup">Signup</Link>
87 </p>
88 </div>
89 </div>
90 );
91 };
92
93 export default Login;
```

src/pages/Login.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Login.jsx
2  +++ after/src/pages/Login.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import authStyles from "../styles/auth.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -20,6 +22,7 @@
10   const [password, setPassword] = useState("");
11
12   const [error, setError] = useState("");
13  + const [fieldErrors, setFieldErrors] = useState({});
14
15   const [loading, setLoading] = useState(false);
16
17  @@ -27,6 +30,7 @@
18   event.preventDefault();
19
20   setError("");
21  + setFieldErrors({});
22   setLoading(true);
23
24   try {
25  @@ -34,9 +38,10 @@
26
27     navigate("/");
28   } catch (error) {
29  +   setFieldErrors(apiError(error).fieldErrors);
30     console.error(error);
31
32  -   setError(error.response?.data?.message || "Login failed");
33  +   setError(apiErrorMessage(error, "Login failed"));
34   } finally {
35     setLoading(false);
36   }
37  @@ -50,7 +55,7 @@
38   <h2>Login</h2>
39
40   <form onSubmit={handleSubmit}>
41  -   <FormField label="Email" autoComplete="email"
42  +   <FormField label="Email" error={fieldErrors.email} autoComplete="email"
43     type="email"
44     placeholder="Email"
45     value={email}
46  @@ -58,7 +63,7 @@
47     required
48   />
49
50  -   <FormField label="Password" autoComplete="current-password"
51  +   <FormField label="Password" error={fieldErrors.password} autoComplete="current-password"
52     type="password"
53     placeholder="Password"
54     value={password}
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import chatStyles from "../styles/chat.module.css";
3  import { bindStyles } from "../utils/bindStyles";
4  import StatusMessage from "../components/ui/StatusMessage";
5  import { useCallback, useEffect, useRef, useState } from "react";
6
7  import { useSearchParams } from "react-router-dom";
8
9  import Navbar from "../components/layout/Navbar";
10
11 import ConversationList from "../components/chat/ConversationList";
12 import ChatWindow from "../components/chat/ChatWindow";
13
14 import {
15   getConversations,
16   getConversation,
17   getOrCreateConversation,
18 } from "../api/conversationApi";
19
20 import { useAuth } from "../auth/AuthContext";
21
22 import { createConversationUpdateWebSocket } from "../services/conversationUpdateWebSocketService";
23 import { subscribeConversationSidebar } from "../services/conversationSidebarSubscription";
24 import { applyConversationUpdate, mergeConversation, mergeConversationLists } from "../utils/conversationUpdates"
25 > ;
26
27 const css = bindStyles(chatStyles);
28
29 const Messages = () => {
30   const { user } = useAuth();
31
32   const [searchParams, setSearchParams] = useSearchParams();
33
34   const [conversations, setConversations] = useState([]);
35
36   const [selectedConversation, setSelectedConversation] = useState(null);
37
38   const [loading, setLoading] = useState(true);
39
40   const [error, setError] = useState("");
41   const conversationsRef = useRef(conversations);
42   useEffect(() => {
43     conversationsRef.current = conversations;
44   }, [conversations]);
45
46   /*
47    * Desktop always shows both panels.
48    * On mobile this controls whether we are currently
49    * looking at the conversation list or the chat window.
50    */
51   const [mobileView, setMobileView] = useState("conversations");
52
53   // =====
54   // LOAD CONVERSATIONS
55   // =====
56
57   const loadConversations = useCallback(async () => {
58     try {
59       setLoading(true);
60       setError("");
61
62       const data = await getConversations();
63
64       setConversations((current) => mergeConversationLists(current, data || []));
65
66       return data || [];
67     } catch (error) {
68       console.error("Failed to load conversations:", error);
69     }

```

```

70     setError(apiErrorMessage(error, "Unable to load conversations"));
71
72     return [];
73   } finally {
74     setLoading(false);
75   }
76 }, []);
77
78 // =====
79 // INITIAL LOAD
80 // =====
81
82 useEffect(() => {
83   let cancelled = false;
84
85   const initialize = async () => {
86     const data = await loadConversations();
87
88     if (cancelled) {
89       return;
90     }
91
92     const requestedUserId = searchParams.get("userId");
93
94     if (!requestedUserId) {
95       if (data.length > 0) {
96         setSelectedConversation((current) => current || data[0]);
97       }
98
99       return;
100     }
101
102     try {
103       const conversation = await getOrCreateConversation(requestedUserId);
104
105       if (cancelled) {
106         return;
107       }
108
109       const refreshed = await loadConversations();
110
111       if (cancelled) {
112         return;
113       }
114
115       const matchingConversation = refreshed.find(
116         (item) => String(item.id) === String(conversation.id),
117       );
118
119       const resolvedConversation = matchingConversation || conversation;
120
121       setSelectedConversation(resolvedConversation);
122
123       /*
124        * When Messages is opened directly through
125        * /messages?userId=..., mobile should go
126        * directly into the chat.
127        */
128       setMobileView("chat");
129
130       setSearchParams(
131         {},
132         {
133           replace: true,
134         },
135       );
136     } catch (error) {
137       if (!cancelled) {
138         console.error("Failed to open conversation:", error);
139       }

```

```

140         setError(
141             apiErrorMessage(error, "Unable to start conversation"),
142         );
143     }
144 }
145 };
146
147 initialize();
148
149 return () => {
150     cancelled = true;
151 };
152 }, [loadConversations, searchParams, setSearchParams]);
153
154 // =====
155 // SELECT CONVERSATION
156 // =====
157
158 const handleSelectConversation = (conversation) => {
159     setSelectedConversation(conversation);
160
161     /*
162      * On mobile, selecting a conversation should
163      * switch from the conversation list to chat.
164      *
165      * On desktop this state has no visual effect.
166      */
167     setMobileView("chat");
168 };
169
170 // =====
171 // BACK TO CONVERSATIONS
172 // =====
173
174 const handleBackToConversations = () => {
175     setMobileView("conversations");
176 };
177
178 // =====
179 // HANDLE REALTIME MESSAGE
180 // =====
181
182 const handleConversationUpdate = useCallback((update) => {
183     if (update?.conversationId == null || !update.lastMessage) return;
184     setConversations((current) => applyConversationUpdate(current, update));
185     setSelectedConversation((current) =>
186         current && String(current.id) === String(update.conversationId)
187             ? mergeConversation(current, {
188                 ...current,
189                 lastMessage: update.lastMessage,
190                 updatedAt: update.updatedAt || update.lastMessage.createdAt,
191             })
192             : current,
193     );
194 }, []);
195
196 const handleMessageReceived = useCallback((message) => {
197     if (message?.conversationId == null) return;
198     handleConversationUpdate({
199         conversationId: message.conversationId,
200         lastMessage: message,
201         updatedAt: message.createdAt,
202     });
203 }, [handleConversationUpdate]);
204
205 useEffect(() => {
206     if (!user?.id) return;
207     return subscribeConversationSidebar({
208         createSocket: createConversationUpdateWebSocket,
209         loadList: getConversations,

```

```

210     loadConversation: getConversation,
211     hasConversation: (id) =>
212       conversationsRef.current.some((row) => String(row.id) === id),
213     onUpdate: handleConversationUpdate,
214     onRows: (rows) => {
215       setConversations((current) => mergeConversationLists(current, rows));
216       setSelectedConversation((current) => {
217         const incoming = rows.find((row) => String(row.id) === String(current?.id));
218         return current && incoming ? mergeConversation(current, incoming) : current;
219       });
220     },
221     onError: (error) => {
222       console.error("Conversation sidebar update failed:", error);
223     },
224   });
225 }, [user?.id, handleConversationUpdate]);
226
227 // =====
228 // LOADING
229 // =====
230
231 if (loading && conversations.length === 0) {
232   return (
233     <>
234       <Navbar />
235
236       <div id="main-content" tabIndex=-1 role="status" className={css("loading-screen")}>Loading messages...<
237     > /div>
238     </>
239   );
240 }
241
242 return (
243   <>
244     <Navbar />
245
246     <main id="main-content" tabIndex=-1 className={css("messages-page")}>
247       <div className={css("messages-card")}>
248         {error && <StatusMessage tone="error" className={css("error chat-page-error")}>{error}</StatusMessage>}
249
250         <div
251           className={css(`messages-layout ${
252             mobileView === "chat"
253               ? "mobile-chat-active"
254               : "mobile-conversations-active"
255           })}
256         >
257           <ConversationList
258             conversations={conversations}
259             selectedConversationId={selectedConversation?.id}
260             onSelect={handleSelectConversation}
261             loading={loading}
262           />
263
264           <ChatWindow
265             conversation={selectedConversation}
266             currentUserId={user?.id}
267             onMessageReceived={handleMessageReceived}
268             onBackToConversations={handleBackToConversations}
269           />
270         </div>
271       </div>
272     </main>
273   </>
274 );
275
276 export default Messages;

```

src/pages/Messages.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Messages.jsx
2  +++ after/src/pages/Messages.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../utils/apiError.js";
5  import chatStyles from "../styles/chat.module.css";
6  import { bindStyles } from "../utils/bindStyles";
7  import StatusMessage from "../components/ui/StatusMessage";
8  @@ -66,7 +67,7 @@
9      } catch (error) {
10         console.error("Failed to load conversations:", error);
11
12 -         setError(error.response?.data?.message || "Unable to load conversations");
13 +         setError(apiErrorMessage(error, "Unable to load conversations"));
14
15         return [];
16     } finally {
17 @@ -137,7 +138,7 @@
18         console.error("Failed to open conversation:", error);
19
20         setError(
21 -         error.response?.data?.message || "Unable to start conversation",
22 +         apiErrorMessage(error, "Unable to start conversation"),
23     );
24     }
25 }
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import notificationsStyles from "../styles/notifications.module.css";
3  import { bindStyles } from "../utils/bindStyles";
4  import StatusMessage from "../components/ui/StatusMessage";
5  import PageContainer from "../components/ui/PageContainer";
6  import Card from "../components/ui/Card";
7  import { useEffect, useState } from "react";
8
9  import Navbar from "../components/layout/Navbar";
10
11 import { useNotifications } from "../context/NotificationContext";
12
13 const css = bindStyles(notificationsStyles);
14
15 const PAGE_SIZE = 10;
16
17 const Notifications = () => {
18   const {
19     notifications,
20     loading,
21     error,
22     loadNotifications,
23     markAsRead,
24     markAllAsRead,
25   } = useNotifications();
26
27   const [page, setPage] = useState(0);
28
29   const [totalPages, setTotalPages] = useState(0);
30
31   const [actionLoadingId, setActionLoadingId] = useState(null);
32
33   const [markAllLoading, setMarkAllLoading] = useState(false);
34
35   const [actionError, setActionError] = useState("");
36
37   // =====
38   // LOAD PAGE
39   // =====
40
41   const loadPage = async (nextPage) => {
42     try {
43       setActionError("");
44
45       const data = await loadNotifications(nextPage, PAGE_SIZE);
46
47       setPage(data?.number ?? nextPage);
48       setTotalPages(data?.totalPages ?? 0);
49     } catch (error) {
50       console.error("Failed to load notification page:", error);
51     }
52   };
53
54   // =====
55   // INITIAL LOAD
56   // =====
57
58   useEffect(() => {
59     loadPage(0);
60   }, []);
61
62   // =====
63   // MARK ONE AS READ
64   // =====
65
66   const handleMarkAsRead = async (notificationId) => {
67     try {
68       setActionError("");
69       setActionLoadingId(notificationId);
70

```

```

71     await markAsRead(notificationId);
72   } catch (error) {
73     console.error("Failed to mark notification as read:", error);
74
75     setActionError(
76       apiErrorMessage(error, "Unable to mark notification as read"),
77     );
78   } finally {
79     setActionLoadingId(null);
80   }
81 };
82
83 // =====
84 // MARK ALL AS READ
85 // =====
86
87 const handleMarkAllAsRead = async () => {
88   try {
89     setActionError("");
90     setMarkAllLoading(true);
91
92     await markAllAsRead();
93   } catch (error) {
94     console.error("Failed to mark all notifications as read:", error);
95
96     setActionError(
97       apiErrorMessage(error, "Unable to mark all notifications as read"),
98     );
99   } finally {
100     setMarkAllLoading(false);
101   }
102 };
103
104 // =====
105 // LOADING
106 // =====
107
108 if (loading && notifications.length === 0) {
109   return (
110     <>
111       <Navbar />
112
113       <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading notification
114     </div>
115   );
116 }
117
118 // =====
119 // UI
120 // =====
121
122 return (
123   <>
124     <Navbar />
125
126     <PageContainer className={css("notifications-page")}>
127       <Card className={css("notifications-card")}>
128         <div className={css("notifications-header")}>
129           <div>
130             <h1>Notifications</h1>
131
132             <p>Stay up to date with your FrndBook activity.</p>
133           </div>
134
135           {notifications.some((notification) => !notification.read) && (
136             <button
137               type="button"
138               className={css("notifications-mark-all")}
139               onClick={handleMarkAllAsRead}

```

```

140         disabled={markAllLoading}
141       >
142         {markAllLoading ? "Marking..." : "Mark all as read"}
143       </button>
144     )}
145   </div>
146
147   {(error || actionError) && (
148     <StatusMessage tone="error" className={css("error")}>{actionError || error}</StatusMessage>
149   )}
150
151   {notifications.length === 0 && !error && (
152     <div className={css("notifications-empty")}>
153       <h2>No notifications yet</h2>
154
155       <p>You're all caught up. New activity will appear here.</p>
156     </div>
157   )}
158
159   {notifications.length > 0 && (
160     <div className={css("notification-list")}>
161       {notifications.map((notification) => (
162         <article
163           key={notification.id}
164           className={css(`notification-item ${
165             notification.read
166               ? "notification-read"
167               : "notification-unread"
168           })`}
169         >
170           <div className={css("notification-indicator")}>
171             {!notification.read && (
172               <span
173                 className={css("notification-unread-dot")}
174                 aria-label="Unread"
175               />
176             )}
177           </div>
178
179           <div className={css("notification-content")}>
180             <p className={css("notification-message")}>
181               {notification.message}
182             </p>
183
184             <div className={css("notification-meta")}>
185               <span>
186                 {notification.createdAt
187                   ? new Date(notification.createdAt).toLocaleString()
188                   : ""}
189               </span>
190
191               {notification.type && (
192                 <span className={css("notification-type")}>
193                   {notification.type}
194                 </span>
195               )}
196             </div>
197           </div>
198
199           {!notification.read && (
200             <button
201               type="button"
202               className={css("notification-read-button")}
203               onClick={() => handleMarkAsRead(notification.id)}
204               disabled={actionLoadingId === notification.id}
205             >
206               {actionLoadingId === notification.id
207                 ? "..."
208                 : "Mark read"}
209             </button>

```

```
210         })
211       </article>
212     )}
213   </div>
214 )}
215
216   {totalPages > 1 && (
217     <div className={css("notification-pagination")}>
218       <button
219         type="button"
220         disabled={loading || page === 0}
221         onClick={() => loadPage(page - 1)}
222       >
223         Previous
224       </button>
225
226       <span>
227         Page {page + 1} of {totalPages}
228       </span>
229
230       <button
231         type="button"
232         disabled={loading || page >= totalPages - 1}
233         onClick={() => loadPage(page + 1)}
234       >
235         Next
236       </button>
237     </div>
238   )}
239 </Card>
240 </PageContainer>
241 </>
242 );
243 };
244
245 export default Notifications;
```

src/pages/Notifications.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/pages/Notifications.jsx
2 +++ after/src/pages/Notifications.jsx
3 @@ -1,3 +1,4 @@
4 +import { apiErrorMessage } from "../utils/apiError.js";
5 import notificationsStyles from "../styles/notifications.module.css";
6 import { bindStyles } from "../utils/bindStyles";
7 import StatusMessage from "../components/ui/StatusMessage";
8 @@ -72,7 +73,7 @@
9     console.error("Failed to mark notification as read:", error);
10
11     setActionError(
12 -     error.response?.data?.message || "Unable to mark notification as read",
13 +     apiErrorMessage(error, "Unable to mark notification as read"),
14     );
15   } finally {
16     setActionLoadingId(null);
17 @@ -93,8 +94,7 @@
18     console.error("Failed to mark all notifications as read:", error);
19
20     setActionError(
21 -     error.response?.data?.message ||
22 -     "Unable to mark all notifications as read",
23 +     apiErrorMessage(error, "Unable to mark all notifications as read"),
24     );
25   } finally {
26     setMarkAllLoading(false);
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import profileStyles from "../styles/profile.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import PageContainer from "../components/ui/PageContainer";
8  import Card from "../components/ui/Card";
9  import { useEffect, useRef, useState } from "react";
10
11 import Navbar from "../components/layout/Navbar";
12
13 import UserAvatar from "../components/users/UserAvatar";
14
15 import { updateProfile, updateProfileImage } from "../api/userApi";
16
17 import { useAuth } from "../auth/AuthContext";
18
19 const css = bindStyles(profileStyles);
20
21 const ProfileForm = () => {
22   const { user, loading: authLoading, updateUser, sessionKey } = useAuth();
23   const profile = user;
24   const busy = useRef(false);
25   const mounted = useRef(false);
26   useEffect(() => { mounted.current = true; return () => { mounted.current = false; }; }, []);
27
28   const [nameDraft, setName] = useState(null);
29   const name = nameDraft ?? user?.name ?? "";
30
31   const [bioDraft, setBio] = useState(null);
32   const bio = bioDraft ?? user?.bio ?? "";
33
34   const [loading, setLoading] = useState(false);
35
36   const [imageLoading, setImageLoading] = useState(false);
37
38   const [error, setError] = useState("");
39   const [fieldErrors, setFieldErrors] = useState({});
40
41   const [success, setSuccess] = useState("");
42
43   const [previewImage, setPreviewImage] = useState(null);
44
45   const fileInputRef = useRef(null);
46
47   // =====
48   // CLEANUP PREVIEW
49   // =====
50
51   useEffect(() => {
52     return () => {
53       if (previewImage) {
54         URL.revokeObjectURL(previewImage);
55       }
56     };
57   }, [previewImage]);
58
59   // =====
60   // UPDATE PROFILE
61   // =====
62
63   const handleSubmit = async (event) => {
64     event.preventDefault();
65     if (busy.current) return;
66     busy.current = true;
67
68     setError("");
69     setFieldErrors({});
70     setSuccess("");

```

```

71     setLoading(true);
72
73     try {
74         const updatedUser = await updateProfile(name.trim(), bio.trim());
75
76         if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;
77         setName(null);
78         setBio(null);
79
80         setSuccess("Profile updated successfully.");
81     } catch (error) {
82         if (!mounted.current) return;
83         setFieldErrors(apiError(error).fieldErrors);
84         console.error("Profile update failed:", error);
85
86         setError(apiErrorMessage(error, "Failed to update profile"));
87     } finally {
88         busy.current = false;
89         if (mounted.current) setLoading(false);
90     }
91 };
92
93 // =====
94 // IMAGE SELECTION
95 // =====
96
97 const handleImageChange = (event) => {
98     if (busy.current) return;
99     const file = event.target.files?.[0];
100
101     if (!file) {
102         return;
103     }
104
105     if (!["image/jpeg", "image/png", "image/webp"].includes(file.type)) {
106         setError("Please select a JPEG, PNG or WebP image.");
107         return;
108     }
109
110     setError("");
111     setFieldErrors({});
112     setSuccess("");
113
114     if (previewImage) {
115         URL.revokeObjectURL(previewImage);
116     }
117
118     const previewUrl = URL.createObjectURL(file);
119
120     setPreviewImage(previewUrl);
121
122     void handleImageUpload(file);
123 };
124
125 // =====
126 // IMAGE UPLOAD
127 // =====
128
129 const handleImageUpload = async (file) => {
130     busy.current = true;
131     setImageLoading(true);
132
133     setError("");
134     setFieldErrors({});
135     setSuccess("");
136
137     try {
138         const updatedUser = await updateProfileImage(file);
139
140         if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;

```

```

141
142     setSuccess("Profile image updated successfully.");
143
144     setPreviewImage(null);
145   } catch (error) {
146     if (!mounted.current) return;
147     setFieldErrors(apiError(error).fieldErrors);
148     console.error("Profile image upload failed:", error);
149
150     if (previewImage) {
151       URL.revokeObjectURL(previewImage);
152     }
153
154     setPreviewImage(null);
155
156     setError(
157       apiErrorMessage(error, "Failed to upload profile image"),
158     );
159   } finally {
160     busy.current = false;
161     if (mounted.current) setImageLoading(false);
162
163     if (fileInputRef.current) {
164       fileInputRef.current.value = "";
165     }
166   }
167 };
168
169 // =====
170 // LOADING
171 // =====
172
173 if (authLoading) {
174   return <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading...</div>;
175 }
176
177 if (!profile) {
178   return <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Unable to load pr
179 > ofile.</div>;
180 }
181
182 return (
183   <>
184     <Navbar />
185
186     <PageContainer className={css("profile-page")}>
187       <Card className={css("profile-card")}>
188         <h1>My Profile</h1>
189
190         {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
191
192         {success && <StatusMessage tone="success" className={css("success")}>{success}</StatusMessage>}
193
194         <div className={css("profile-image-section")}>
195           <UserAvatar
196             name={profile.name}
197             image={previewImage || profile.profileImage}
198             userId={profile.id}
199             size="large"
200           />
201
202           <button
203             type="button"
204             onClick={() => fileInputRef.current?.click()}
205             disabled={imageLoading || loading}
206             >
207             {imageLoading ? "Uploading..." : "Change Photo"}
208           </button>
209
210           <input

```

```

210         ref={fileInputRef}
211         type="file"
212         accept="image/jpeg,image/png,image/webp"
213         onChange={handleImageChange}
214         hidden
215     />
216 </div>
217
218 <form className={css("profile-form")} onSubmit={handleSubmit}>
219
220     <FormField label="Name" error={fieldErrors.name} autoComplete="name"
221       type="text"
222       disabled={loading || imageLoading}
223       value={name}
224       maxLength={100}
225       onChange={(event) => setName(event.target.value)}
226       required
227     />
228
229     <FormField label="Email" type="email" autoComplete="email" value={profile.email} disabled />
230
231     <FormField as="textarea" label="Bio" error={fieldErrors.bio}
232       disabled={loading || imageLoading}
233       value={bio}
234       maxLength={500}
235       rows={5}
236       onChange={(event) => setBio(event.target.value)}
237     />
238
239     <div className={css("character-count")}>{bio.length}/500</div>
240
241     <button type="submit" disabled={loading || imageLoading}>
242       {loading ? "Saving..." : "Save Changes"}
243     </button>
244 </form>
245
246 <div className={css("profile-details")}>
247   <p>
248     <strong>Status:</strong> {profile.status || "-"}
249   </p>
250
251   <p>
252     <strong>Last seen:</strong>{" "}
253     {profile.lastSeen
254       ? new Date(profile.lastSeen).toLocaleString()
255       : "-"}
256   </p>
257 </div>
258 </Card>
259 </PageContainer>
260 </>
261 );
262 };
263
264 export default function Profile() {
265   const { user, sessionKey } = useAuth();
266   return <ProfileForm key={`_${sessionKey}:${user?.id}`} />;
267 }

```

src/pages/Profile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Profile.jsx
2  +++ after/src/pages/Profile.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import profileStyles from "../styles/profile.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -16,40 +18,31 @@
10
11  const css = bindStyles(profileStyles);
12
13  -const Profile = () => {
14  -  const { user, loading: authLoading } = useAuth();
15  -
16  -  const [profile, setProfile] = useState(user);
17  -
18  -  const [name, setName] = useState(user?.name || "");
19  -
20  -  const [bio, setBio] = useState(user?.bio || "");
21  +const ProfileForm = () => {
22  +  const { user, loading: authLoading, updateUser, sessionKey } = useAuth();
23  +  const profile = user;
24  +  const busy = useRef(false);
25  +  const mounted = useRef(false);
26  +  useEffect(() => { mounted.current = true; return () => { mounted.current = false; }; }, []);
27  +
28  +  const [nameDraft, setName] = useState(null);
29  +  const name = nameDraft ?? user?.name ?? "";
30  +
31  +  const [bioDraft, setBio] = useState(null);
32  +  const bio = bioDraft ?? user?.bio ?? "";
33
34  const [loading, setLoading] = useState(false);
35
36  const [imageLoading, setImageLoading] = useState(false);
37
38  const [error, setError] = useState("");
39  + const [fieldErrors, setFieldErrors] = useState({});
40
41  const [success, setSuccess] = useState("");
42
43  const [previewImage, setPreviewImage] = useState(null);
44
45  const fileInputRef = useRef(null);
46  -
47  - // =====
48  - // SYNC USER
49  - // =====
50  -
51  - useEffect(() => {
52  -   if (!user) {
53  -     return;
54  -   }
55  -
56  -   setProfile(user);
57  -   setName(user.name || "");
58  -   setBio(user.bio || "");
59  - }, [user]);
60
61  // =====
62  // CLEANUP PREVIEW
63  @@ -69,25 +62,31 @@
64
65  const handleSubmit = async (event) => {
66    event.preventDefault();
67  +   if (busy.current) return;
68  +   busy.current = true;
69
70    setError("");
```

```

71 +   setFieldErrors({});
72   setSuccess("");
73   setLoading(true);
74
75   try {
76     const updatedUser = await updateProfile(name.trim(), bio.trim());
77
78 -   setProfile(updatedUser);
79 -   setName(updatedUser.name || "");
80 -   setBio(updatedUser.bio || "");
81 +   if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;
82 +   setName(null);
83 +   setBio(null);
84
85     setSuccess("Profile updated successfully.");
86   } catch (error) {
87 +   if (!mounted.current) return;
88 +   setFieldErrors(apiError(error).fieldErrors);
89     console.error("Profile update failed:", error);
90
91 -   setError(error.response?.data?.message || "Failed to update profile");
92 +   setError(apiErrorMessage(error, "Failed to update profile"));
93   } finally {
94 -   setLoading(false);
95 +   busy.current = false;
96 +   if (mounted.current) setLoading(false);
97   }
98 };
99
100 @@ -96,18 +95,20 @@
101   // =====
102
103   const handleImageChange = (event) => {
104 +   if (busy.current) return;
105     const file = event.target.files?.[0];
106
107     if (!file) {
108       return;
109     }
110
111 -   if (!file.type.startsWith("image/")) {
112 -     setError("Please select an image file.");
113 +   if (!["image/jpeg", "image/png", "image/webp"].includes(file.type)) {
114 +     setError("Please select a JPEG, PNG or WebP image.");
115     return;
116   }
117
118   setError("");
119 +   setFieldErrors({});
120   setSuccess("");
121
122   if (previewImage) {
123 @@ -118,7 +119,7 @@
124
125     setPreviewImage(previewUrl);
126
127 -   handleImageUpload(file);
128 +   void handleImageUpload(file);
129   };
130
131   // =====
132 @@ -126,23 +127,24 @@
133   // =====
134
135   const handleImageUpload = async (file) => {
136 +   busy.current = true;
137     setImageLoading(true);
138
139     setError("");
140 +   setFieldErrors({});

```

src/pages/Profile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
141     setSuccess("");
142
143     try {
144         const updatedUser = await updateProfileImage(file);
145
146 -         setProfile(updatedUser);
147 +         if (!mounted.current || !updateUser(updatedUser, sessionKey)) return;
148
149         setSuccess("Profile image updated successfully.");
150
151 -         /*
152 -         * Keep the local preview visible until
153 -         * the profile page is refreshed.
154 -         */
155 +         setPreviewImage(null);
156     } catch (error) {
157 +         if (!mounted.current) return;
158 +         setFieldErrors(apiError(error).fieldErrors);
159         console.error("Profile image upload failed:", error);
160
161         if (previewImage) {
162 @@ -152,10 +154,11 @@
163             setPreviewImage(null);
164
165             setError(
166 -                 error.response?.data?.message || "Failed to upload profile image",
167 +                 apiErrorMessage(error, "Failed to upload profile image"),
168             );
169         } finally {
170 -             setImageLoading(false);
171 +             busy.current = false;
172 +             if (mounted.current) setImageLoading(false);
173
174             if (fileInputRef.current) {
175                 fileInputRef.current.value = "";
176 @@ -198,7 +201,7 @@
177                 <button
178                     type="button"
179                     onClick={() => fileInputRef.current?.click()}
180 -                     disabled={imageLoading}
181 +                     disabled={imageLoading || loading}
182                 >
183                     {imageLoading ? "Uploading..." : "Change Photo"}
184                 </button>
185 @@ -206,7 +209,7 @@
186                 <input
187                     ref={fileInputRef}
188                     type="file"
189 -                     accept="image/*"
190 +                     accept="image/jpeg,image/png,image/webp"
191                     onChange={handleImageChange}
192                     hidden
193                 />
194 @@ -214,8 +217,9 @@
195
196                 <form className={css("profile-form")} onSubmit={handleSubmit}>
197
198 -                 <FormField label="Name" autoComplete="name"
199 +                 <FormField label="Name" error={fieldErrors.name} autoComplete="name"
200                     type="text"
201 +                     disabled={loading || imageLoading}
202                     value={name}
203                     maxLength={100}
204                     onChange={(event) => setName(event.target.value)}
205 @@ -224,7 +228,8 @@
206
207                 <FormField label="Email" type="email" autoComplete="email" value={profile.email} disabled />
208
209 -                 <FormField as="textarea" label="Bio"
210 +                 <FormField as="textarea" label="Bio" error={fieldErrors.bio}
```

src/pages/Profile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
211 +         disabled={loading || imageLoading}
212         value={bio}
213         maxLength={500}
214         rows={5}
215 @@ -233,7 +238,7 @@
216
217         <div className={css("character-count")}>{bio.length}/500</div>
218
219 -         <button type="submit" disabled={loading}>
220 +         <button type="submit" disabled={loading || imageLoading}>
221             {loading ? "Saving..." : "Save Changes"}
222         </button>
223     </form>
224 @@ -256,4 +261,7 @@
225     );
226 };
227
228 -export default Profile;
229 +export default function Profile() {
230 +   const { user, sessionKey } = useAuth();
231 +   return <ProfileForm key={`_${sessionKey}:${user?.id}`} />;
232 +}
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import authStyles from "../styles/auth.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import Button from "../components/ui/Button";
8  import { useState } from "react";
9  import { Link, useNavigate, useSearchParams } from "react-router-dom";
10
11 import { resetPassword } from "../api/authApi";
12
13 const css = bindStyles(authStyles);
14
15 const ResetPassword = () => {
16   const navigate = useNavigate();
17
18   const [searchParams] = useSearchParams();
19
20   const token = searchParams.get("token") || "";
21
22   const [password, setPassword] = useState("");
23
24   const [confirmPassword, setConfirmPassword] = useState("");
25
26   const [error, setError] = useState("");
27   const [fieldErrors, setFieldErrors] = useState({});
28
29   const [success, setSuccess] = useState("");
30
31   const [loading, setLoading] = useState(false);
32
33   const handleSubmit = async (event) => {
34     event.preventDefault();
35
36     setError("");
37     setFieldErrors({});
38     setSuccess("");
39
40     if (!token) {
41       setError("Invalid or missing reset link.");
42
43       return;
44     }
45
46     if (password !== confirmPassword) {
47       setError("Passwords do not match.");
48
49       return;
50     }
51
52     if (password.length < 8) {
53       setError("Password must be at least 8 characters.");
54
55       return;
56     }
57
58     setLoading(true);
59
60     try {
61       await resetPassword({
62         token,
63         newPassword: password,
64       });
65
66       setSuccess("Password reset successfully. Redirecting to login...");
67
68       window.setTimeout(() => {
69         navigate("/login");
70       }, 1200);

```

```

71     } catch (error) {
72       setFieldErrors(apiError(error).fieldErrors);
73       console.error(error);
74
75       setError(apiErrorMessage(error, "Unable to reset password"));
76     } finally {
77       setLoading(false);
78     }
79   };
80
81   return (
82     <div className={css("auth-page")}>
83       <div className={css("auth-card")}>
84         <h1>FrndBook</h1>
85
86         <h2>Reset Password</h2>
87
88         <form onSubmit={handleSubmit}>
89           <FormField label="New password" error={fieldErrors.newPassword} autoComplete="new-password"
90             type="password"
91             placeholder="New password"
92             value={password}
93             onChange={(event) => setPassword(event.target.value)}
94             minLength={8}
95             required
96           />
97
98           <FormField label="Confirm password" autoComplete="new-password"
99             type="password"
100             placeholder="Confirm password"
101             value={confirmPassword}
102             onChange={(event) => setConfirmPassword(event.target.value)}
103             minLength={8}
104             required
105           />
106
107           {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
108
109           {success && <StatusMessage tone="success" className={css("success")}>{success}</StatusMessage>}
110
111           <Button type="submit" disabled={loading}>
112             {loading ? "Resetting..." : "Reset Password"}
113           </Button>
114         </form>
115
116         <p>
117           <Link to="/login">Back to Login</Link>
118         </p>
119       </div>
120     </div>
121   );
122 };
123
124 export default ResetPassword;

```

src/pages/ResetPassword.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/ResetPassword.jsx
2  +++ after/src/pages/ResetPassword.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import authStyles from "../styles/auth.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -22,6 +24,7 @@
10   const [confirmPassword, setConfirmPassword] = useState("");
11
12   const [error, setError] = useState("");
13  + const [fieldErrors, setFieldErrors] = useState({});
14
15   const [success, setSuccess] = useState("");
16
17  @@ -31,6 +34,7 @@
18   event.preventDefault();
19
20   setError("");
21  + setFieldErrors({});
22   setSuccess("");
23
24   if (!token) {
25  @@ -65,9 +69,10 @@
26     navigate("/login");
27   }, 1200);
28   } catch (error) {
29  +   setFieldErrors(apiError(error).fieldErrors);
30     console.error(error);
31
32  -   setError(error.response?.data?.message || "Unable to reset password");
33  +   setError(apiErrorMessage(error, "Unable to reset password"));
34   } finally {
35     setLoading(false);
36   }
37  @@ -81,7 +86,7 @@
38     <h2>Reset Password</h2>
39
40     <form onSubmit={handleSubmit}>
41  -     <FormField label="New password" autoComplete="new-password"
42  +     <FormField label="New password" error={fieldErrors.newPassword} autoComplete="new-password"
43         type="password"
44         placeholder="New password"
45         value={password}>
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import authStyles from "../styles/auth.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import Button from "../components/ui/Button";
8  import { useState } from "react";
9  import { Link, useNavigate } from "react-router-dom";
10
11 import { useAuth } from "../auth/AuthContext";
12
13 const css = bindStyles(authStyles);
14
15 const Signup = () => {
16   const navigate = useNavigate();
17
18   const { signup } = useAuth();
19
20   const [name, setName] = useState("");
21   const [email, setEmail] = useState("");
22   const [password, setPassword] = useState("");
23
24   const [error, setError] = useState("");
25   const [fieldErrors, setFieldErrors] = useState({});
26   const [loading, setLoading] = useState(false);
27
28   const handleSubmit = async (event) => {
29     event.preventDefault();
30
31     setError("");
32     setFieldErrors({});
33     setLoading(true);
34
35     try {
36       await signup(name, email, password);
37
38       navigate(`/verify-email?email=${encodeURIComponent(email)}`);
39     } catch (error) {
40       setFieldErrors(apiError(error).fieldErrors);
41       console.error(error);
42
43       setError(
44         apiErrorMessage(error, "Unable to send verification code"),
45       );
46     } finally {
47       setLoading(false);
48     }
49   };
50
51   return (
52     <div className={css("auth-page")}>
53       <div className={css("auth-card")}>
54         <h1>FrndBook</h1>
55
56         <h2>Create Account</h2>
57
58         <form onSubmit={handleSubmit}>
59           <FormField label="Name" error={fieldErrors.name} autoComplete="name"
60             type="text"
61             placeholder="Name"
62             value={name}
63             onChange={(event) => setName(event.target.value)}
64             required
65           />
66
67           <FormField label="Email" error={fieldErrors.email} autoComplete="email"
68             type="email"
69             placeholder="Email"
70             value={email}

```

```
71         onChange={(event) => setEmail(event.target.value)}
72         required
73       />
74
75       <FormField label="Password" error={fieldErrors.password} autoComplete="new-password"
76         type="password"
77         placeholder="Password"
78         value={password}
79         onChange={(event) => setPassword(event.target.value)}
80         minLength={8}
81         required
82       />
83
84       {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}
85
86       <Button type="submit" disabled={loading}>
87         {loading ? "Sending code..." : "Create Account"}
88       </Button>
89     </form>
90
91     <p>
92       Already have an account? <Link to="/login">Login</Link>
93     </p>
94   </div>
95 </div>
96   );
97 };
98
99 export default Signup;
```

src/pages/Signup.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/Signup.jsx
2  +++ after/src/pages/Signup.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import authStyles from "../styles/auth.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -20,12 +22,14 @@
10   const [password, setPassword] = useState("");
11
12   const [error, setError] = useState("");
13  + const [fieldErrors, setFieldErrors] = useState({});
14   const [loading, setLoading] = useState(false);
15
16   const handleSubmit = async (event) => {
17     event.preventDefault();
18
19     setError("");
20  +   setFieldErrors({});
21     setLoading(true);
22
23     try {
24  @@ -33,10 +37,11 @@
25
26     navigate(`/verify-email?email=${encodeURIComponent(email)}`);
27   } catch (error) {
28  +   setFieldErrors(apiError(error).fieldErrors);
29     console.error(error);
30
31     setError(
32  -   error.response?.data?.message || "Unable to send verification code",
33  +   apiErrorMessage(error, "Unable to send verification code"),
34   );
35   } finally {
36     setLoading(false);
37  @@ -51,7 +56,7 @@
38     <h2>Create Account</h2>
39
40     <form onSubmit={handleSubmit}>
41  -   <FormField label="Name" autoComplete="name"
42  +   <FormField label="Name" error={fieldErrors.name} autoComplete="name"
43     type="text"
44     placeholder="Name"
45     value={name}
46  @@ -59,7 +64,7 @@
47     required
48   />
49
50  -   <FormField label="Email" autoComplete="email"
51  +   <FormField label="Email" error={fieldErrors.email} autoComplete="email"
52     type="email"
53     placeholder="Email"
54     value={email}
55  @@ -67,7 +72,7 @@
56     required
57   />
58
59  -   <FormField label="Password" autoComplete="new-password"
60  +   <FormField label="Password" error={fieldErrors.password} autoComplete="new-password"
61     type="password"
62     placeholder="Password"
63     value={password}
```

```

1  import { apiErrorMessage } from "../utils/apiError.js";
2  import friendsStyles from "../styles/friends.module.css";
3  import profileStyles from "../styles/profile.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import PageContainer from "../components/ui/PageContainer";
7  import Card from "../components/ui/Card";
8  import { useEffect, useState } from "react";
9  import { Link, useNavigate, useParams } from "react-router-dom";
10
11 import Navbar from "../components/layout/Navbar";
12
13 import UserAvatar from "../components/users/UserAvatar";
14 import FriendAction from "../components/users/FriendAction";
15
16 import { getUserById } from "../api/userApi";
17
18 import {
19   sendFriendRequest,
20   getFriends,
21   getReceivedFriendRequests,
22   getSentFriendRequests,
23   acceptFriendRequest,
24   rejectFriendRequest,
25   removeFriend,
26 } from "../api/friendApi";
27
28 import { useAuth } from "../auth/AuthContext";
29
30 import { FRIENDSHIP_STATUS, getFriendshipState } from "../utils/friendship";
31
32 const css = bindStyles(friendsStyles, profileStyles);
33
34 const UserProfile = () => {
35   const { userId } = useParams();
36
37   const navigate = useNavigate();
38
39   const { user: currentUser } = useAuth();
40
41   const [user, setUser] = useState(null);
42
43   const [friendshipStatus, setFriendshipStatus] = useState(
44     FRIENDSHIP_STATUS.NONE,
45   );
46
47   const [friendRequestId, setFriendRequestId] = useState(null);
48
49   const [loading, setLoading] = useState(true);
50
51   const [friendshipLoading, setFriendshipLoading] = useState(false);
52
53   const [friendshipLoaded, setFriendshipLoaded] = useState(false);
54
55   const [error, setError] = useState("");
56
57   const [friendshipError, setFriendshipError] = useState("");
58
59   // =====
60   // LOAD PROFILE
61   // =====
62
63   useEffect(() => {
64     const loadUser = async () => {
65       try {
66         setLoading(true);
67         setError("");
68
69         const data = await getUserById(userId);
70

```

```

71     setUser(data);
72   } catch (error) {
73     console.error("Failed to load user profile:", error);
74
75     setError(
76       apiErrorMessage(error, "Unable to load user profile"),
77     );
78   } finally {
79     setLoading(false);
80   }
81 };
82
83 loadUser();
84 }, [userId]);
85
86 // =====
87 // LOAD FRIENDSHIP STATE
88 // =====
89
90 useEffect(() => {
91   if (!user || !currentUser) {
92     return;
93   }
94
95   if (String(user.id) === String(currentUser.id)) {
96     setFriendshipLoaded(true);
97
98     return;
99   }
100
101   const loadFriendshipState = async () => {
102     try {
103       setFriendshipError("");
104       setFriendshipLoaded(false);
105
106       const [friends, receivedRequests, sentRequests] = await Promise.all([
107         getFriends(),
108         getReceivedFriendRequests(),
109         getSentFriendRequests(),
110       ]);
111
112       const state = getFriendshipState(
113         user.id,
114         friends || [],
115         receivedRequests || [],
116         sentRequests || [],
117       );
118
119       setFriendshipStatus(state.status);
120
121       setFriendRequestId(state.requestId);
122     } catch (error) {
123       console.error("Failed to load friendship state:", error);
124
125       setFriendshipError(
126         apiErrorMessage(error, "Unable to load friendship status"),
127       );
128     } finally {
129       setFriendshipLoaded(true);
130     }
131   };
132
133   loadFriendshipState();
134 }, [user, currentUser]);
135
136 // =====
137 // ADD FRIEND
138 // =====
139
140 const handleAddFriend = async () => {

```

```

141     if (!user) {
142         return;
143     }
144
145     try {
146         setFriendshipLoading(true);
147         setFriendshipError("");
148
149         const request = await sendFriendRequest(user.id);
150
151         setFriendshipStatus(FRIENDSHIP_STATUS.REQUEST_SENT);
152
153         setFriendRequestId(request?.id || null);
154     } catch (error) {
155         console.error("Failed to send friend request:", error);
156
157         setFriendshipError(
158             apiErrorMessage(error, "Unable to send friend request"),
159         );
160     } finally {
161         setFriendshipLoading(false);
162     }
163 };
164
165 // =====
166 // ACCEPT
167 // =====
168
169 const handleAcceptFriend = async () => {
170     if (!friendRequestId) {
171         return;
172     }
173
174     try {
175         setFriendshipLoading(true);
176         setFriendshipError("");
177
178         await acceptFriendRequest(friendRequestId);
179
180         setFriendshipStatus(FRIENDSHIP_STATUS.FRIENDS);
181
182         setFriendRequestId(null);
183     } catch (error) {
184         console.error("Failed to accept friend request:", error);
185
186         setFriendshipError(
187             apiErrorMessage(error, "Unable to accept friend request"),
188         );
189     } finally {
190         setFriendshipLoading(false);
191     }
192 };
193
194 // =====
195 // REJECT
196 // =====
197
198 const handleRejectFriend = async () => {
199     if (!friendRequestId) {
200         return;
201     }
202
203     try {
204         setFriendshipLoading(true);
205         setFriendshipError("");
206
207         await rejectFriendRequest(friendRequestId);
208
209         setFriendshipStatus(FRIENDSHIP_STATUS.NONE);
210

```

```

211     setFriendRequestId(null);
212   } catch (error) {
213     console.error("Failed to reject friend request:", error);
214
215     setFriendshipError(
216       apiErrorMessage(error, "Unable to reject friend request"),
217     );
218   } finally {
219     setFriendshipLoading(false);
220   }
221 };
222
223 // =====
224 // REMOVE FRIEND
225 // =====
226
227 const handleRemoveFriend = async () => {
228   if (!user) {
229     return;
230   }
231
232   const confirmed = window.confirm(
233     `Are you sure you want to remove ${user.name} from your friends?`,
234   );
235
236   if (!confirmed) {
237     return;
238   }
239
240   try {
241     setFriendshipLoading(true);
242     setFriendshipError("");
243
244     await removeFriend(user.id);
245
246     setFriendshipStatus(FRIENDSHIP_STATUS.NONE);
247   } catch (error) {
248     console.error("Failed to remove friend:", error);
249
250     setFriendshipError(
251       apiErrorMessage(error, "Unable to remove friend"),
252     );
253   } finally {
254     setFriendshipLoading(false);
255   }
256 };
257
258 // =====
259 // OPEN MESSAGE
260 // =====
261
262 const handleMessage = () => {
263   if (!user) {
264     return;
265   }
266
267   /*
268    * Reuse the existing Messages page flow.
269    *
270    * Messages.jsx already handles:
271    *
272    * /messages?userId={userId}
273    *
274    * and resolves/opens the conversation using
275    * getOrCreateConversation().
276    */
277   navigate(`/messages?userId=${user.id}`);
278 };
279
280 // =====

```

```

281 // LOADING
282 // =====
283
284 if (loading) {
285   return (
286     <>
287       <Navbar />
288
289       <div id="main-content" tabIndex={-1} role="status" className={css("loading-screen")}>Loading profile...</div>
290     </>
291   );
292 }
293
294 // =====
295 // PROFILE ERROR
296 // =====
297
298 if (error || !user) {
299   return (
300     <>
301       <Navbar />
302
303       <PageContainer className={css("profile-page")}>
304         <Card className={css("profile-card")}>
305           <StatusMessage tone="error" className={css("error")}>{error || "User not found."}</StatusMessage>
306
307           <Link to="/" className={css("back-link")}>
308             ← Back to Home
309           </Link>
310         </Card>
311       </PageContainer>
312     </>
313   );
314 }
315
316 const isOwnProfile = String(user.id) === String(currentUser?.id);
317
318 // =====
319 // UI
320 // =====
321
322 return (
323   <>
324     <Navbar />
325
326     <PageContainer className={css("profile-page")}>
327       <Card className={css("profile-card user-profile-card")}>
328         <Link to="/" className={css("back-link")}>
329           ← Back to Home
330         </Link>
331
332         <div className={css("profile-image-section")}>
333           <UserAvatar
334             name={user.name}
335             image={user.profileImage}
336             userId={user.id}
337             size="large"
338           />
339         </div>
340
341         <div className={css("user-profile-info")}>
342           <h1>{user.name}</h1>
343
344           <p className={css("user-profile-bio")}>{user.bio || "No bio available"}</p>
345
346           <div className={css("profile-details")}>
347             <p>
348               <strong>Status:</strong> {user.status || "-"}
349             </p>

```

```

350
351         <p>
352             <strong>Last seen:</strong>{" "}
353             {user.lastSeen ? new Date(user.lastSeen).toLocaleString() : "-"}
354         </p>
355     </div>
356 </div>
357
358     {!isOwnProfile && (
359     <div className={css("friend-action-section")}>
360         {friendshipError && <StatusMessage tone="error" className={css("error")}>{friendshipError}</StatusM
361     > message>}
362
363         {!friendshipLoaded ? (
364             <p className={css("friendship-loading")}>Checking friendship...</p>
365         ) : (
366             <FriendAction
367                 status={friendshipStatus}
368                 onAdd={handleAddFriend}
369                 onAccept={handleAcceptFriend}
370                 onReject={handleRejectFriend}
371                 onRemove={handleRemoveFriend}
372                 onMessage={handleMessage}
373                 loading={friendshipLoading}
374             </>
375         )}
376     </div>
377 </Card>
378 </PageContainer>
379 </>
380 );
381 };
382
383 export default UserProfile;

```

src/pages/UserProfile.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/UserProfile.jsx
2  +++ after/src/pages/UserProfile.jsx
3  @@ -1,3 +1,4 @@
4  +import { apiErrorMessage } from "../utils/apiError.js";
5  import friendsStyles from "../styles/friends.module.css";
6  import profileStyles from "../styles/profile.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  @@ -72,7 +73,7 @@
9      console.error("Failed to load user profile:", error);
10
11      setError(
12 -      error.response?.data?.message || "Unable to load user profile",
13 +      apiErrorMessage(error, "Unable to load user profile"),
14      );
15      } finally {
16          setLoading(false);
17  @@ -122,7 +123,7 @@
18      console.error("Failed to load friendship state:", error);
19
20      setFriendshipError(
21 -      error.response?.data?.message || "Unable to load friendship status",
22 +      apiErrorMessage(error, "Unable to load friendship status"),
23      );
24      } finally {
25          setFriendshipLoaded(true);
26  @@ -154,7 +155,7 @@
27      console.error("Failed to send friend request:", error);
28
29      setFriendshipError(
30 -      error.response?.data?.message || "Unable to send friend request",
31 +      apiErrorMessage(error, "Unable to send friend request"),
32      );
33      } finally {
34          setFriendshipLoading(false);
35  @@ -183,7 +184,7 @@
36      console.error("Failed to accept friend request:", error);
37
38      setFriendshipError(
39 -      error.response?.data?.message || "Unable to accept friend request",
40 +      apiErrorMessage(error, "Unable to accept friend request"),
41      );
42      } finally {
43          setFriendshipLoading(false);
44  @@ -212,7 +213,7 @@
45      console.error("Failed to reject friend request:", error);
46
47      setFriendshipError(
48 -      error.response?.data?.message || "Unable to reject friend request",
49 +      apiErrorMessage(error, "Unable to reject friend request"),
50      );
51      } finally {
52          setFriendshipLoading(false);
53  @@ -247,7 +248,7 @@
54      console.error("Failed to remove friend:", error);
55
56      setFriendshipError(
57 -      error.response?.data?.message || "Unable to remove friend",
58 +      apiErrorMessage(error, "Unable to remove friend"),
59      );
60      } finally {
61          setFriendshipLoading(false);
```

```

1  import { apiError } from "../utils/apiError";
2  import { apiErrorMessage } from "../utils/apiError.js";
3  import authStyles from "../styles/auth.module.css";
4  import { bindStyles } from "../utils/bindStyles";
5  import StatusMessage from "../components/ui/StatusMessage";
6  import FormField from "../components/ui/FormField";
7  import Button from "../components/ui/Button";
8  import { useEffect, useState } from "react";
9  import { Link, useNavigate, useSearchParams } from "react-router-dom";
10
11 import { resendVerification, verifyEmail } from "../api/authApi";
12
13 const css = bindStyles(authStyles);
14
15 const RESEND_COOLDOWN_SECONDS = 60;
16
17 const VerifyEmail = () => {
18   const navigate = useNavigate();
19
20   const [searchParams] = useSearchParams();
21
22   const [email, setEmail] = useState(searchParams.get("email") || "");
23
24   const [otp, setOtp] = useState("");
25
26   const [error, setError] = useState("");
27   const [fieldErrors, setFieldErrors] = useState({});
28   const [success, setSuccess] = useState("");
29
30   const [loading, setLoading] = useState(false);
31   const [resending, setResending] = useState(false);
32
33   const [resendCountdown, setResendCountdown] = useState(
34     RESEND_COOLDOWN_SECONDS,
35   );
36
37   useEffect(() => {
38     if (resendCountdown <= 0) {
39       return;
40     }
41
42     const timer = window.setInterval(() => {
43       setResendCountdown((current) => Math.max(0, current - 1));
44     }, 1000);
45
46     return () => {
47       window.clearInterval(timer);
48     };
49   }, [resendCountdown]);
50
51   const handleVerify = async (event) => {
52     event.preventDefault();
53
54     setError("");
55     setFieldErrors({});
56     setSuccess("");
57     setLoading(true);
58
59     try {
60       await verifyEmail({
61         email,
62         otp,
63       });
64
65       setSuccess("Email verified successfully. Redirecting to login...");
66
67       window.setTimeout(() => {
68         navigate("/login");
69       }, 1200);
70     } catch (error) {

```

```

71     setFieldErrors(apiError(error).fieldErrors);
72     console.error(error);
73
74     setError(apiErrorMessage(error, "Unable to verify email"));
75   } finally {
76     setLoading(false);
77   }
78 };
79
80 const handleResend = async () => {
81   if (resendCountdown > 0) {
82     return;
83   }
84
85   setError("");
86   setFieldErrors({});
87   setSuccess("");
88   setResending(true);
89
90   try {
91     await resendVerification({
92       email,
93     });
94
95     setSuccess("A new verification code has been sent.");
96
97     setResendCountdown(RESEND_COOLDOWN_SECONDS);
98   } catch (error) {
99     setFieldErrors(apiError(error).fieldErrors);
100    console.error(error);
101
102    setError(
103      apiErrorMessage(error, "Unable to resend verification code"),
104    );
105   } finally {
106     setResending(false);
107   }
108 };
109
110 return (
111   <div className={css("auth-page")}>
112     <div className={css("auth-card")}>
113       <h1>FrndBook</h1>
114
115       <h2>Verify Your Email</h2>
116
117       <p>Enter the 6-digit verification code sent to your email.</p>
118
119       <form onSubmit={handleVerify}>
120         <FormField label="Email" error={fieldErrors.email} autoComplete="email"
121           type="email"
122           placeholder="Email"
123           value={email}
124           onChange={(event) => setEmail(event.target.value)}
125           required
126         />
127
128         <FormField label="Verification code" error={fieldErrors.otp} autoComplete="one-time-code"
129           type="text"
130           inputMode="numeric"
131           maxLength={6}
132           placeholder="6-digit code"
133           value={otp}
134           onChange={(event) =>
135             setOtp(event.target.value.replace(/\D/g, "").slice(0, 6))
136           }
137           required
138         />
139
140         {error && <StatusMessage tone="error" className={css("error")}>{error}</StatusMessage>}

```

```
141
142     {success && <StatusMessage tone="success" className={css("success")}>{success}</StatusMessage>}
143
144     <Button type="submit" disabled={loading || otp.length !== 6}>
145       {loading ? "Verifying..." : "Verify Email"}
146     </Button>
147   </form>
148
149   <Button
150     type="button"
151     onClick={handleResend}
152     disabled={resending || resendCountdown > 0}
153   >
154     {resending
155       ? "Sending..."
156       : resendCountdown > 0
157         ? `Resend code in ${resendCountdown}s`
158         : "Resend code"}
159   </Button>
160
161   <p>
162     <Link to="/login">Back to Login</Link>
163   </p>
164 </div>
165 </div>
166 );
167 };
168
169 export default VerifyEmail;
```

src/pages/VerifyEmail.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/pages/VerifyEmail.jsx
2  +++ after/src/pages/VerifyEmail.jsx
3  @@ -1,3 +1,5 @@
4  +import { apiError } from "../utils/apiError";
5  +import { apiErrorMessage } from "../utils/apiError.js";
6  import authStyles from "../styles/auth.module.css";
7  import { bindStyles } from "../utils/bindStyles";
8  import StatusMessage from "../components/ui/StatusMessage";
9  @@ -22,6 +24,7 @@
10   const [otp, setOtp] = useState("");
11
12   const [error, setError] = useState("");
13  + const [fieldErrors, setFieldErrors] = useState({});
14   const [success, setSuccess] = useState("");
15
16   const [loading, setLoading] = useState(false);
17  @@ -49,6 +52,7 @@
18   event.preventDefault();
19
20   setError("");
21  + setFieldErrors({});
22   setSuccess("");
23   setLoading(true);
24
25  @@ -64,9 +68,10 @@
26   navigate("/login");
27   }, 1200);
28   } catch (error) {
29  +   setFieldErrors(apiError(error).fieldErrors);
30   console.error(error);
31
32  -   setError(error.response?.data?.message || "Unable to verify email");
33  +   setError(apiErrorMessage(error, "Unable to verify email"));
34   } finally {
35     setLoading(false);
36   }
37  @@ -78,6 +83,7 @@
38   }
39
40   setError("");
41  + setFieldErrors({});
42   setSuccess("");
43   setResending(true);
44
45  @@ -90,10 +96,11 @@
46
47   setResendCountdown(RESEND_COOLDOWN_SECONDS);
48   } catch (error) {
49  +   setFieldErrors(apiError(error).fieldErrors);
50   console.error(error);
51
52   setError(
53  -   error.response?.data?.message || "Unable to resend verification code",
54  +   apiErrorMessage(error, "Unable to resend verification code"),
55   );
56   } finally {
57     setResending(false);
58  @@ -110,7 +117,7 @@
59     <p>Enter the 6-digit verification code sent to your email.</p>
60
61     <form onSubmit={handleVerify}>
62  -     <FormField label="Email" autoComplete="email"
63  +     <FormField label="Email" error={fieldErrors.email} autoComplete="email"
64         type="email"
65         placeholder="Email"
66         value={email}
67  @@ -118,7 +125,7 @@
68         required
69     />
70
```

src/pages/VerifyEmail.jsx

UNIFIED DIFF: + ADDED / - REMOVED

```
71 -      <FormField label="Verification code" autoComplete="one-time-code"
72 +      <FormField label="Verification code" error={fieldErrors.otp} autoComplete="one-time-code"
73         type="text"
74         inputMode="numeric"
75         maxLength={6}
```

```

1 import { apiErrorMessage } from "../utils/apiError.js";
2 import { compareMessages, lastPage, mergeMessages } from "../utils/messageHistory.js";
3
4 export const MESSAGE_PAGE_SIZE = 10;
5 const MAX_SCAN_PAGES = 1000;
6
7 /** One lifetime per account/conversation. Dependencies make async races testable. */
8 export function createMessageSession({ conversationId, fetchPage, saveMessage, markRead,
9   isVisible = () => true, onState, onMessage }) {
10   let alive = true;
11   let initialized = false;
12   let anchor = null;
13   let olderPage = 0;
14   let historyFlight = null;
15   let recoverQueued = false;
16   let sendFlight = false;
17   let readTimer = null;
18   let reading = false;
19   let readAgain = false;
20   const abort = new AbortController();
21   let state = { messages: [], loading: true, syncing: false, loadingOlder: false,
22     hasOlder: false, historyError: "", sendError: "" };
23
24   const update = (patch) => {
25     if (!alive) return;
26     state = { ...state, ...patch };
27     onState(state);
28   };
29   const accept = (messages) => {
30     if (!alive) return;
31     update({ messages: mergeMessages(state.messages, messages, conversationId) });
32   };
33   const requestRead = () => {
34     if (!alive || !initialized || !isVisible()) return;
35     if (reading) { readAgain = true; return; }
36     if (readTimer) return;
37     readTimer = setTimeout(async () => {
38       readTimer = null;
39       if (!alive || !isVisible()) return;
40       reading = true;
41       try { await markRead(); } catch { /* Read failure must not hide delivered messages. */ }
42       finally {
43         reading = false;
44         if (readAgain) { readAgain = false; requestRead(); }
45       }
46     }, 100);
47   };
48   const load = async (page) => {
49     const data = await fetchPage(page, MESSAGE_PAGE_SIZE, abort.signal);
50     if (!data || !Array.isArray(data.content)) throw new Error("Invalid message history response");
51     return data;
52   };
53   const runHistory = (kind) => {
54     if (!alive) return Promise.resolve();
55     if (historyFlight) {
56       if (kind !== "older") recoverQueued = true;
57       return historyFlight;
58     }
59     const initial = !initialized;
60     update({ historyError: "", loading: initial, syncing: !initial && kind !== "older", loadingOlder: kind === "o
61 > lder" });
62     historyFlight = (async () => {
63       try {
64         if (initial) {
65           const data = await load(0);
66           if (!alive) return;
67           accept(data.content);
68           anchor = mergeMessages([], data.content, conversationId).at(-1) || null;
69           olderPage = 0;
70           initialized = true;

```

```

70     update({ hasOlder: !lastPage(data, 0, MESSAGE_PAGE_SIZE) });
71   } else {
72     const oldest = state.messages[0];
73     const boundary = kind === "older" ? oldest : anchor;
74     let nextAnchor = null;
75     let page = kind === "older" ? Math.max(0, olderPage - 1) : 0;
76     let complete = false;
77     for (let count = 0; count < MAX_SCAN_PAGES && alive; count++, page++) {
78       const data = await load(page);
79       if (!alive) return;
80       const incoming = mergeMessages([], data.content, conversationId);
81       if (kind !== "older" && page === 0) nextAnchor = incoming.at(-1) || anchor;
82       accept(incoming);
83       const end = lastPage(data, page, MESSAGE_PAGE_SIZE);
84       const reached = kind === "older"
85         ? incoming.some(message => !boundary || compareMessages(message, boundary) < 0)
86         : incoming.some(message => boundary && String(message.id) === String(boundary.id));
87       if (reached || end) {
88         if (kind === "older") {
89           olderPage = page;
90           update({ hasOlder: !end });
91         } else {
92           anchor = nextAnchor;
93           if (end) update({ hasOlder: false });
94           else if (!oldest) update({ hasOlder: true });
95         }
96         complete = true;
97         break;
98       }
99     }
100     if (alive && !complete) throw new Error("History is too large to synchronize in one pass. Reopen this c
> onversation to load recent messages.");
101   }
102   if (alive) {
103     const latest = state.messages.at(-1);
104     if (latest) onMessage?.(latest);
105     requestRead();
106   }
107 } catch (error) {
108   if (alive) update({ historyError: error.response?.data?.message || error.message || "Unable to load messa
> ges" });
109 } finally {
110   historyFlight = null;
111   if (alive) {
112     update({ loading: false, syncing: false, loadingOlder: false });
113     if (recoverQueued) { recoverQueued = false; void runHistory("recover"); }
114   }
115 }
116 })();
117 return historyFlight;
118 };
119
120 return {
121   initialize: () => runHistory("initial"),
122   recover: () => runHistory("recover"),
123   loadOlder: () => state.hasOlder ? runHistory("older") : Promise.resolve(),
124   requestRead,
125   receive(message) {
126     if (!alive || message?.id == null || String(message.conversationId) !== String(conversationId)) return;
127     accept([message]);
128     onMessage?.(message);
129     requestRead();
130   },
131   async send(content) {
132     if (!alive || sendFlight) throw new Error("A message is already being sent");
133     sendFlight = true;
134     update({ sendError: "" });
135     try {
136       // REST provides a persistence acknowledgement. WebSocket remains live delivery.
137       const saved = await saveMessage(content);

```

```
138         if (saved?.id == null || String(saved.conversationId) !== String(conversationId)) {
139             throw new Error("The server did not confirm this message");
140         }
141         if (alive) { accept([saved]); onMessage?.(saved); }
142         return saved;
143     } catch (error) {
144         const uncertain = "Send could not be confirmed. Your text was kept. Check recent messages before retrying
145     > .";
146         update({ sendError: error.response ? apiErrorMessage(error, uncertain) : uncertain });
147         throw error;
148     } finally { sendFlight = false; }
149 },
150 dispose() {
151     alive = false;
152     abort.abort();
153     clearTimeout(readTimer);
154 },
155 };
```

src/services/messageSession.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- before/src/services/messageSession.js
2 +++ after/src/services/messageSession.js
3 @@ -1,3 +1,4 @@
4 +import { apiErrorMessage } from "../utils/apiError.js";
5 import { compareMessages, lastPage, mergeMessages } from "../utils/messageHistory.js";
6
7 export const MESSAGE_PAGE_SIZE = 10;
8 @@ -140,8 +141,8 @@
9     if (alive) { accept([saved]); onMessage?.(saved); }
10     return saved;
11   } catch (error) {
12 -     update({ sendError: error.response?.data?.message ||
13 -       "Send could not be confirmed. Your text was kept. Check recent messages before retrying." });
14 +     const uncertain = "Send could not be confirmed. Your text was kept. Check recent messages before retryin
15 > g.";
16 +     update({ sendError: error.response ? apiErrorMessage(error, uncertain) : uncertain });
17     throw error;
18   } finally { sendFlight = false; }
19 },
```

```
1  /** Accept the backend's public error contract, never raw Axios/internal errors. */
2  export function apiError(error, fallback = "Unable to complete this request.") {
3    const data = error?.response?.data;
4    const fieldErrors = Object.fromEntries(Object.entries(
5      data?.fieldErrors && typeof data.fieldErrors === "object" && !Array.isArray(data.fieldErrors)
6        ? data.fieldErrors : {},
7    ).filter(([ , value]) => typeof value === "string" && value.trim()));
8    const status = error?.response?.status;
9    const message = typeof data?.message === "string" && data.message.trim()
10      ? data.message : error?.code === "ERR_NETWORK"
11        ? "Unable to reach the server. Check your connection and try again."
12        : status === 429 ? "Too many requests. Please wait before trying again." : fallback;
13    return { message, fieldErrors };
14  }
15
16  export function apiErrorMessage(error, fallback) {
17    const { message, fieldErrors } = apiError(error, fallback);
18    const details = [...new Set(Object.values(fieldErrors))];
19    return [message, ...details.filter(detail => detail !== message)].join(" ");
20  }
```

```
1 --- /dev/null
2 +++ after/src/utils/apiError.js
3 @@ -0,0 +1,20 @@
4 +/** Accept the backend's public error contract, never raw Axios/internal errors. */
5 +export function apiError(error, fallback = "Unable to complete this request.") {
6 +  const data = error?.response?.data;
7 +  const fieldErrors = Object.fromEntries(Object.entries(
8 +    data?.fieldErrors && typeof data.fieldErrors === "object" && !Array.isArray(data.fieldErrors)
9 +      ? data.fieldErrors : {},
10 +  ).filter([, value] => typeof value === "string" && value.trim()));
11 +  const status = error?.response?.status;
12 +  const message = typeof data?.message === "string" && data.message.trim()
13 +    ? data.message : error?.code === "ERR_NETWORK"
14 +      ? "Unable to reach the server. Check your connection and try again."
15 +      : status === 429 ? "Too many requests. Please wait before trying again." : fallback;
16 +  return { message, fieldErrors };
17 +}
18 +
19 +export function apiErrorMessage(error, fallback) {
20 +  const { message, fieldErrors } = apiError(error, fallback);
21 +  const details = [...new Set(Object.values(fieldErrors))];
22 +  return [message, ...details.filter(detail => detail !== message)].join(" ");
23 +}
```

```

1  const ACCESS_TOKEN_KEY = "frndbook_access_token";
2  const REFRESH_TOKEN_KEY = "frndbook_refresh_token";
3  const USER_KEY = "frndbook_user";
4  const SESSION_KEY = "frndbook_session";
5
6  export const TOKEN_UPDATED_EVENT = "frndbook:tokens-updated";
7
8  export const AUTH_CLEARED_EVENT = "frndbook:auth-cleared";
9
10 // =====
11 // EVENTS
12 // =====
13
14 const dispatchTokenUpdated = () => {
15   window.dispatchEvent(new Event(TOKEN_UPDATED_EVENT));
16 };
17
18 const dispatchAuthCleared = () => {
19   window.dispatchEvent(new Event(AUTH_CLEARED_EVENT));
20 };
21
22 // =====
23 // TOKEN STORAGE
24 // =====
25
26 export const tokenStorage = {
27   getSessionId() { return localStorage.getItem(SESSION_KEY); },
28   // =====
29   // ACCESS TOKEN
30   // =====
31
32   getAccessToken() {
33     return localStorage.getItem(ACCESS_TOKEN_KEY);
34   },
35
36   // =====
37   // REFRESH TOKEN
38   // =====
39
40   getRefreshToken() {
41     return localStorage.getItem(REFRESH_TOKEN_KEY);
42   },
43
44   // =====
45   // USER
46   // =====
47
48   getUser() {
49     const user = localStorage.getItem(USER_KEY);
50
51     if (!user) {
52       return null;
53     }
54
55     try {
56       return JSON.parse(user);
57     } catch {
58       localStorage.removeItem(USER_KEY);
59
60       return null;
61     }
62   },
63
64   // =====
65   // SAVE AUTH
66   // =====
67
68   saveAuth(accessToken, refreshToken, user) {
69     localStorage.setItem(SESSION_KEY, crypto.randomUUID());
70     localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);

```

```

71
72     localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
73
74     localStorage.setItem(USER_KEY, JSON.stringify(user));
75
76     dispatchTokenUpdated();
77 },
78
79 // =====
80 // UPDATE ACCESS TOKEN
81 // =====
82
83 updateAccessToken(accessToken) {
84     if (!accessToken) {
85         return;
86     }
87
88     localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);
89
90     dispatchTokenUpdated();
91 },
92
93 // =====
94 // UPDATE ACCESS + REFRESH TOKEN
95 // =====
96
97 updateTokens(accessToken, refreshToken) {
98     if (!accessToken) {
99         return;
100     }
101
102     localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);
103
104     if (refreshToken) {
105         localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
106     }
107
108     dispatchTokenUpdated();
109 },
110
111 // =====
112 // UPDATE USER
113 // =====
114
115 updateUser(user) {
116     const serialized = JSON.stringify(user);
117     if (localStorage.getItem(USER_KEY) !== serialized) localStorage.setItem(USER_KEY, serialized);
118 },
119
120 // =====
121 // CLEAR AUTH
122 // =====
123
124 clear() {
125     localStorage.removeItem(SESSION_KEY);
126     localStorage.removeItem(ACCESS_TOKEN_KEY);
127
128     localStorage.removeItem(REFRESH_TOKEN_KEY);
129
130     localStorage.removeItem(USER_KEY);
131
132     dispatchAuthCleared();
133 },
134 };

```

src/utils/tokenStorage.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1  --- before/src/utils/tokenStorage.js
2  +++ after/src/utils/tokenStorage.js
3  @@ -1,6 +1,7 @@
4     const ACCESS_TOKEN_KEY = "frndbook_access_token";
5     const REFRESH_TOKEN_KEY = "frndbook_refresh_token";
6     const USER_KEY = "frndbook_user";
7  +const SESSION_KEY = "frndbook_session";
8
9     export const TOKEN_UPDATED_EVENT = "frndbook:tokens-updated";
10
11  @@ -23,6 +24,7 @@
12     // =====
13
14     export const tokenStorage = {
15  +  getSessionId() { return localStorage.getItem(SESSION_KEY); },
16     // =====
17     // ACCESS TOKEN
18     // =====
19  @@ -64,6 +66,7 @@
20     // =====
21
22     saveAuth(accessToken, refreshToken, user) {
23  +  localStorage.setItem(SESSION_KEY, crypto.randomUUID());
24     localStorage.setItem(ACCESS_TOKEN_KEY, accessToken);
25
26     localStorage.setItem(REFRESH_TOKEN_KEY, refreshToken);
27  @@ -110,7 +113,8 @@
28     // =====
29
30     updateUser(user) {
31  -  localStorage.setItem(USER_KEY, JSON.stringify(user));
32  +  const serialized = JSON.stringify(user);
33  +  if (localStorage.getItem(USER_KEY) !== serialized) localStorage.setItem(USER_KEY, serialized);
34     },
35
36     // =====
37  @@ -118,6 +122,7 @@
38     // =====
39
40     clear() {
41  +  localStorage.removeItem(SESSION_KEY);
42     localStorage.removeItem(ACCESS_TOKEN_KEY);
43
44     localStorage.removeItem(REFRESH_TOKEN_KEY);
```

```

1 import { createServer } from "../node_modules/vite/dist/node/index.js";
2 import { fileURLToPath } from "node:url";
3 import assert from "node:assert/strict";
4 const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
5 const origin="http://127.0.0.1:41817";
6 process.env.VITE_API_BASE_URL=origin;process.env.VITE_WS_URL=origin.replace("http","ws")+"/ws";
7 const server=await createServer({root:fileURLToPath(new URL("../import.meta.url")),server:{host:"127.0.0.1",port:
> 41817,strictPort:true}});
8 const errors=[];
9 let account={id:1,name:"Original",email:"person@example.invalid",bio:"Saved bio",profileImage:null};
10 let unavailable=false,rejectProfile=false,imageFailure=false,holdProfile=false,heldRoute;
11 let expired=false,refreshStatus=503;
12 let browser;
13 try {
14   await server.listen();browser=await chromium.launch({channel:"msedge",headless:true});
15   const context=await browser.newContext();
16   await context.route("**/*",route=>{
17     const url=new URL(route.request().url());assert.equal(url.origin,origin);
18     if(!url.pathname.startsWith("/api/"))return route.continue();
19     if(url.pathname==="/api/users/me") {
20       if(route.request().method==="PATCH") {
21         if(holdProfile){heldRoute=route;return;}
22         if(rejectProfile)return route.fulfill({status:400,json:{message:"Validation failed",fieldErrors:{name:"Na
> me rejected"}}});
23         account={...account,...route.request().postDataJSON()};
24       } else if(expired)return route.fulfill({status:401,json:{message:"Expired"}});
25       else if(unavailable)return route.fulfill({status:503,json:{message:"Temporary outage"}});
26       return route.fulfill({json:account});
27     }
28     if(url.pathname.endsWith("profile-image")) {
29       if(imageFailure)return route.fulfill({status:400,json:{message:"Image rejected"}});
30       account={...account,profileImage:origin+"/fixture-avatar.png"};return route.fulfill({json:account});
31     }
32     if(url.pathname==="/api/auth/signup")return route.fulfill({status:400,json:{message:"Validation failed",field
> Errors:{email:"Email rejected",password:"Password rejected"}}});
33     if(url.pathname==="/api/auth/refresh")return route.fulfill({status:refreshStatus,json:{message:"Refresh unava
> ilable"}});
34     if(url.pathname.endsWith("unread-count"))return route.fulfill({json:{}});
35     return route.fulfill({json:[]});
36   });
37   await context.routeWebSocket("**/*",socket=>socket.onMessage(frame=>{
38     if(/^(\CONNECT|STOMP)/.test(String(frame)))socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
39     if(String(frame).startsWith("DISCONNECT"))socket.close();
40   }));
41   const page=await context.newPage();page.on("pageerror",e=>errors.push(e.message));
42   await page.goto(origin+"/login");
43   await page.evaluate(async account=>{
44     const {tokenStorage}=await import("/src/utils/tokenStorage.js");
45     tokenStorage.saveAuth("test."+btoa(JSON.stringify({exp:999999999}))+".test","refresh",account);
46   },account);
47   await page.goto(origin+"/profile");
48   await page.getByLabel("Name",{exact:true}).fill("Updated");
49   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
50   await page.getByText("Profile updated successfully",{exact:true}).waitFor();
51   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).name),"Updated");
52   await page.getByRole("link",{name:"Home",exact:true}).click();
53   await page.getByText("Hello, Updated",{exact:true}).waitFor();
54   const second=await context.newPage();second.on("pageerror",e=>errors.push(e.message));
55   await second.goto(origin+"/");await second.getByText("Hello, Updated",{exact:true}).waitFor();
56   await page.goto(origin+"/profile");await page.getByLabel("Name",{exact:true}).fill("Across tabs");
57   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
58   await second.getByText("Hello, Across tabs",{exact:true}).waitFor();
59   rejectProfile=true;await page.getByLabel("Name",{exact:true}).fill("Invalid name");
60   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
61   await page.locator('input[aria-invalid="true"]').waitFor();
62   assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
63   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).name),"Across tabs");
64   rejectProfile=false;
65   // Failed image upload retains the saved avatar and unsaved text draft.
66   imageFailure=true;

```

```

67   await page.locator('input[type="file"]').setInputFiles({name:"fixture.png",mimeType:"image/png",buffer:Buffer.f
  > rom("fixture")}));
68   await page.getByRole("alert").filter({hasText:"Image rejected"}).waitFor();
69   assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
70   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).profileImage),null);
71   // Successful image response updates shared state without replacing a text draft.
72   imageFailure=false;
73   await page.locator('input[type="file"]').setInputFiles({name:"fixture.png",mimeType:"image/png",buffer:Buffer.f
  > rom("fixture")}));
74   await page.getByText("Profile image updated successfully",{exact:true}).waitFor();
75   assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
76   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).profileImage),origin+"/f
  > ixture-avatar.png");
77   // Clear credentials while a profile save is pending; late success cannot restore it.
78   holdProfile=true;await page.getByRole("button",{name:"Save Changes",exact:true}).click();
79   for(let i=0;!heldRoute&&i<200;i++)await new Promise(r=>setTimeout(r,10));assert.ok(heldRoute);
80   await second.evaluate(async()=>{(await import("/src/utils/tokenStorage.js")).tokenStorage.clear();});
81   await page.waitForURL("*/login");await second.waitForURL("*/login");
82   await heldRoute.fulfill({json:{...account,name:"Stale saved result"}});holdProfile=false;
83   assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_user")),null);
84   // Actual signup fields display backend errors with accessible field associations.
85   await page.goto(origin+"/signup");await page.getByLabel("Name",{exact:true}).fill("Name");
86   await page.getByLabel("Email",{exact:true}).fill("person@example.invalid");await page.getByLabel("Password",{ex
  > act:true}).fill("password123");
87   await page.getByRole("button",{name:"Create Account",exact:true}).click();
88   await page.getByLabel("Email",{exact:true}).locator('xpath=..').getByText("Email rejected",{exact:true}).waitFo
  > r();
89   assert.equal(await page.getByLabel("Password",{exact:true}).getAttribute("aria-invalid"),"true");
90   // Startup outage: keep credentials, expose retry, recover using the same session.
91   await page.evaluate(async account=>{(await import("/src/utils/tokenStorage.js")).tokenStorage.saveAuth("token",
  > "refresh",account);},account);
92   unavailable=true;await page.goto(origin+"/");await page.getByRole("button",{name:"Retry session check"}).waitFo
  > r();
93   assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),"token");
94   unavailable=false;await page.getByRole("button",{name:"Retry session check"}).click();await page.getByText("Hel
  > lo, Across tabs",{exact:true}).waitFor();
95   expired=true;await page.goto(origin+"/");await page.getByRole("button",{name:"Retry session check"}).waitFor();
96   assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),"token");
97   expired=false;await page.getByRole("button",{name:"Retry session check"}).click();await page.getByText("Hello,
  > Across tabs",{exact:true}).waitFor();
98   refreshStatus=401;expired=true;await page.goto(origin+"/");await page.waitForURL("*/login");
99   assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),null);
100  assert.deepEqual(errors,[]);
101  console.log("PASS: shared profile/home/storage, cross-tab profile and logout, failed/successful images preserve
  > drafts, inline validation, stale save after logout, transient startup/refresh retry, rejected refresh clears aut
  > h.");
102 } finally {await browser?.close();await server.close();}

```

tests/accountState.browser.mjs

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- /dev/null
2 +++ after/tests/accountState.browser.mjs
3 @@ -0,0 +1,102 @@
4 +import { createServer } from "../node_modules/vite/dist/node/index.js";
5 +import { fileURLToPath } from "node:url";
6 +import assert from "node:assert/strict";
7 +const { chromium } = await import(process.env.PLAYWRIGHT_MODULE || "playwright");
8 +const origin="http://127.0.0.1:41817";
9 +process.env.VITE_API_BASE_URL=origin;process.env.VITE_WS_URL=origin.replace("http","ws")+"/ws";
10 +const server=await createServer({root:fileURLToPath(new URL("../",import.meta.url)),server:{host:"127.0.0.1",port
  > :41817,strictPort:true}});
11 +const errors=[];
12 +let account={id:1,name:"Original",email:"person@example.invalid",bio:"Saved bio",profileImage:null};
13 +let unavailable=false,rejectProfile=false,imageFailure=false,holdProfile=false,heldRoute;
14 +let expired=false,refreshStatus=503;
15 +let browser;
16 +try {
17 +   await server.listen();browser=await chromium.launch({channel:"msedge",headless:true});
18 +   const context=await browser.newContext();
19 +   await context.route("**/*",route=>{
20 +     const url=new URL(route.request().url());assert.equal(url.origin,origin);
21 +     if(!url.pathname.startsWith("/api/"))return route.continue();
22 +     if(url.pathname==="/api/users/me") {
23 +       if(route.request().method()=== "PATCH") {
24 +         if(holdProfile){heldRoute=route;return;}
25 +         if(rejectProfile)return route.fulfill({status:400,json:{message:"Validation failed",fieldErrors:{name:"N
  > ame rejected"}}});
26 +         account={...account,...route.request().postDataJSON()};
27 +       } else if(expired)return route.fulfill({status:401,json:{message:"Expired"}});
28 +       else if(unavailable)return route.fulfill({status:503,json:{message:"Temporary outage"}});
29 +       return route.fulfill({json:account});
30 +     }
31 +     if(url.pathname.endsWith("profile-image")) {
32 +       if(imageFailure)return route.fulfill({status:400,json:{message:"Image rejected"}});
33 +       account={...account,profileImage:origin+"/fixture-avatar.png"};return route.fulfill({json:account});
34 +     }
35 +     if(url.pathname==="/api/auth/signup")return route.fulfill({status:400,json:{message:"Validation failed",fiel
  > dErrors:{email:"Email rejected",password:"Password rejected"}}});
36 +     if(url.pathname==="/api/auth/refresh")return route.fulfill({status:refreshStatus,json:{message:"Refresh unav
  > ailable"}});
37 +     if(url.pathname.endsWith("unread-count"))return route.fulfill({json:0});
38 +     return route.fulfill({json:[]});
39 +   });
40 +   await context.routeWebSocket("**/*",socket=>socket.onMessage(frame=>{
41 +     if(/^ (CONNECT|STOMP) /.test(String(frame)))socket.send("CONNECTED\nversion:1.2\nheart-beat:0,0\n\n0");
42 +     if(String(frame).startsWith("DISCONNECT"))socket.close();
43 +   }));
44 +   const page=await context.newPage();page.on("pageerror",e=>errors.push(e.message));
45 +   await page.goto(origin+"/login");
46 +   await page.evaluate(async account=>{
47 +     const {tokenStorage}=await import("/src/utils/tokenStorage.js");
48 +     tokenStorage.saveAuth("test."+btoa(JSON.stringify({exp:9999999999}))+".test","refresh",account);
49 +   },account);
50 +   await page.goto(origin+"/profile");
51 +   await page.getByLabel("Name",{exact:true}).fill("Updated");
52 +   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
53 +   await page.getByText("Profile updated successfully",{exact:true}).waitFor();
54 +   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).name),"Updated");
55 +   await page.getByRole("link",{name:"Home",exact:true}).click();
56 +   await page.getByText("Hello, Updated",{exact:true}).waitFor();
57 +   const second=await context.newPage();second.on("pageerror",e=>errors.push(e.message));
58 +   await second.goto(origin+"/");await second.getByText("Hello, Updated",{exact:true}).waitFor();
59 +   await page.goto(origin+"/profile");await page.getByLabel("Name",{exact:true}).fill("Across tabs");
60 +   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
61 +   await second.getByText("Hello, Across tabs",{exact:true}).waitFor();
62 +   rejectProfile=true;await page.getByLabel("Name",{exact:true}).fill("Invalid name");
63 +   await page.getByRole("button",{name:"Save Changes",exact:true}).click();
64 +   await page.locator('input[aria-invalid="true"]').waitFor();
65 +   assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
66 +   assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).name),"Across tabs");
```

```

67 + rejectProfile=false;
68 + // Failed image upload retains the saved avatar and unsaved text draft.
69 + imageFailure=true;
70 + await page.locator('input[type="file"]').setInputFiles({name:"fixture.png",mimeType:"image/png",buffer:Buffer.
  > from("fixture")});
71 + await page.getByRole("alert").filter({hasText:"Image rejected"}).waitFor();
72 + assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
73 + assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).profileImage),null);
74 + // Successful image response updates shared state without replacing a text draft.
75 + imageFailure=false;
76 + await page.locator('input[type="file"]').setInputFiles({name:"fixture.png",mimeType:"image/png",buffer:Buffer.
  > from("fixture")});
77 + await page.getByText("Profile image updated successfully.",{exact:true}).waitFor();
78 + assert.equal(await page.getByLabel("Name",{exact:true}).inputValue(),"Invalid name");
79 + assert.equal(await page.evaluate(()=>JSON.parse(localStorage.getItem("frndbook_user")).profileImage),origin+"/
  > fixture-avatar.png");
80 + // Clear credentials while a profile save is pending; late success cannot restore it.
81 + holdProfile=true;await page.getByRole("button",{name:"Save Changes",exact:true}).click();
82 + for(let i=0;!heldRoute&&i<200;i++)await new Promise(r=>setTimeout(r,10));assert.ok(heldRoute);
83 + await second.evaluate(async()=>{(await import("/src/utils/tokenStorage.js")).tokenStorage.clear();});
84 + await page.waitForURL("*/login");await second.waitForURL("*/login");
85 + await heldRoute.fulfill({json:{...account,name:"Stale saved result"}});holdProfile=false;
86 + assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_user"),null);
87 + // Actual signup fields display backend errors with accessible field associations.
88 + await page.goto(origin+"/signup");await page.getByLabel("Name",{exact:true}).fill("Name");
89 + await page.getByLabel("Email",{exact:true}).fill("person@example.invalid");await page.getByLabel("Password",{e
  > xact:true}).fill("password123");
90 + await page.getByRole("button",{name:"Create Account",exact:true}).click();
91 + await page.getByLabel("Email",{exact:true}).locator('xpath=..').getByText("Email rejected",{exact:true}).waitF
  > or();
92 + assert.equal(await page.getByLabel("Password",{exact:true}).getAttribute("aria-invalid"),"true");
93 + // Startup outage: keep credentials, expose retry, recover using the same session.
94 + await page.evaluate(async()=>{(await import("/src/utils/tokenStorage.js")).tokenStorage.saveAuth("token"
  > ,"refresh",account);},account);
95 + unavailable=true;await page.goto(origin+"/");await page.getByRole("button",{name:"Retry session check"}).waitF
  > or();
96 + assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),"token");
97 + unavailable=false;await page.getByRole("button",{name:"Retry session check"}).click();await page.getByText("He
  > llo, Across tabs",{exact:true}).waitFor();
98 + expired=true;await page.goto(origin+"/");await page.getByRole("button",{name:"Retry session check"}).waitFor()
  > ;
99 + assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),"token");
100 + expired=false;await page.getByRole("button",{name:"Retry session check"}).click();await page.getByText("Hello,
  > Across tabs",{exact:true}).waitFor();
101 + refreshStatus=401;expired=true;await page.goto(origin+"/");await page.waitForURL("*/login");
102 + assert.equal(await page.evaluate(()=>localStorage.getItem("frndbook_access_token"),null);
103 + assert.deepEqual(errors,[]);
104 + console.log("PASS: shared profile/home/storage, cross-tab profile and logout, failed/successful images preserv
  > e drafts, inline validation, stale save after logout, transient startup/refresh retry, rejected refresh clears au
  > th.");
105 +} finally {await browser?.close();await server.close();}

```

```
1 import test from "node:test";
2 import assert from "node:assert/strict";
3 import { apiError, apiErrorMessage } from "../src/utils/apiError.js";
4
5 test("backend fields remain available and summary includes unmapped fields", () => {
6   const error={response:{data:{message:"Validation failed",fieldErrors:{name:"Name required",other:"Invalid selec
> tion"}}}};
7   assert.equal(apiError(error).fieldErrors.name,"Name required");
8   assert.equal(apiErrorMessage(error),"Validation failed Name required Invalid selection");
9 });
10 test("malformed responses use fallback and never stringify objects or expose internal errors", () => {
11   const error={message:"SQL secret",response:{data:{message:{secret:1},fieldErrors:{a:{secret:1},b:null,c:""}}}};
12   assert.deepEqual(apiError(error,"Try again"),{message:"Try again",fieldErrors:{}});
13   assert.equal(apiErrorMessage({response:{data:"<html>proxy</html>"}}, "Try again"), "Try again");
14 });
15 test("network and rate-limit feedback is actionable", () => {
16   assert.match(apiErrorMessage({code:"ERR_NETWORK"}),/connection/);
17   assert.match(apiErrorMessage({response:{status:429}}),/wait/);
18 });
```

tests/apiError.test.js

UNIFIED DIFF: + ADDED / - REMOVED

```
1 --- /dev/null
2 +++ after/tests/apiError.test.js
3 @@ -0,0 +1,18 @@
4 +import test from "node:test";
5 +import assert from "node:assert/strict";
6 +import { apiError, apiErrorMessage } from "../src/utils/apiError.js";
7 +
8 +test("backend fields remain available and summary includes unmapped fields", () => {
9 +  const error={response:{data:{message:"Validation failed",fieldErrors:{name:"Name required",other:"Invalid selection"}}}};
10 +  assert.equal(apiError(error).fieldErrors.name,"Name required");
11 +  assert.equal(apiErrorMessage(error),"Validation failed Name required Invalid selection");
12 +});
13 +test("malformed responses use fallback and never stringify objects or expose internal errors", () => {
14 +  const error={message:"SQL secret",response:{data:{message:{secret:1},fieldErrors:{a:{secret:1},b:null,c:""}}}}
15 +  ;
16 +  assert.deepEqual(apiError(error,"Try again"),{message:"Try again",fieldErrors:{}});
17 +  assert.equal(apiErrorMessage({response:{data:"<html>proxy</html>"}}, "Try again"), "Try again");
18 +});
19 +test("network and rate-limit feedback is actionable", () => {
20 +  assert.match(apiErrorMessage({code:"ERR_NETWORK"}),/connection/);
21 +  assert.match(apiErrorMessage({response:{status:429}}),/wait/);
22 +});
```